

Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning

Outline

- Introduction
- Related works
- Algorithm details
- Experiments and Results
- Conclusion

Outline

- Introduction
- Related works
- Algorithm details
- Experiments and Results
- Conclusion

Introduction



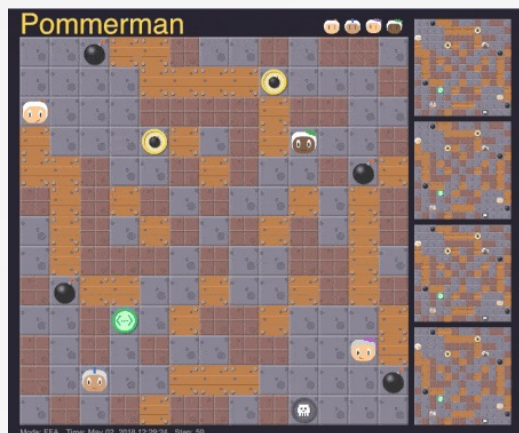
- StarCraft, considered to be one of the most challenging Real-Time Strategy (RTS) games and one of the longest-played esports of all time.
- AlphaStar is the first AI to reach the top league of StarCraft without any game restrictions. It uses a combination of general-purpose techniques for **neural network architectures**, **imitation learning**, **reinforcement learning**, and **multi-agent learning**, and uses data from both human and agent games within a diverse league of continually **adapting strategies and counter-strategies**.

Outline

- Introduction
- **Related works**
- Algorithm details
- Experiments and Results
- Conclusion

CASIA

Related works

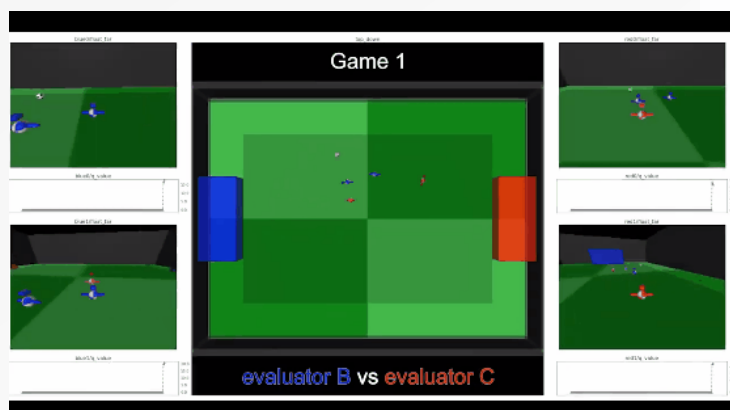


The NeurIPS 2018 Pommernan Competition

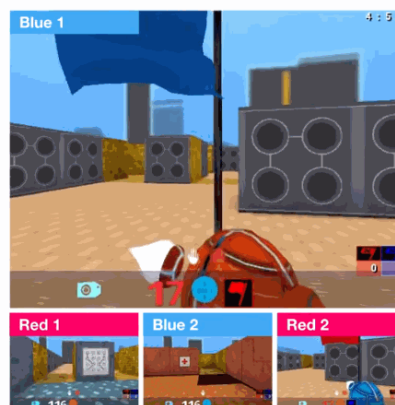


OpenAI Five

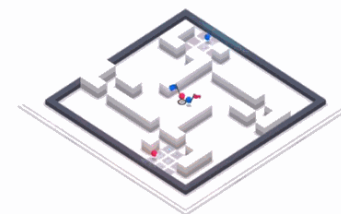
Agent observation raw pixels



Soccer



Quake III Arena Capture the Flag



Indoor map overview

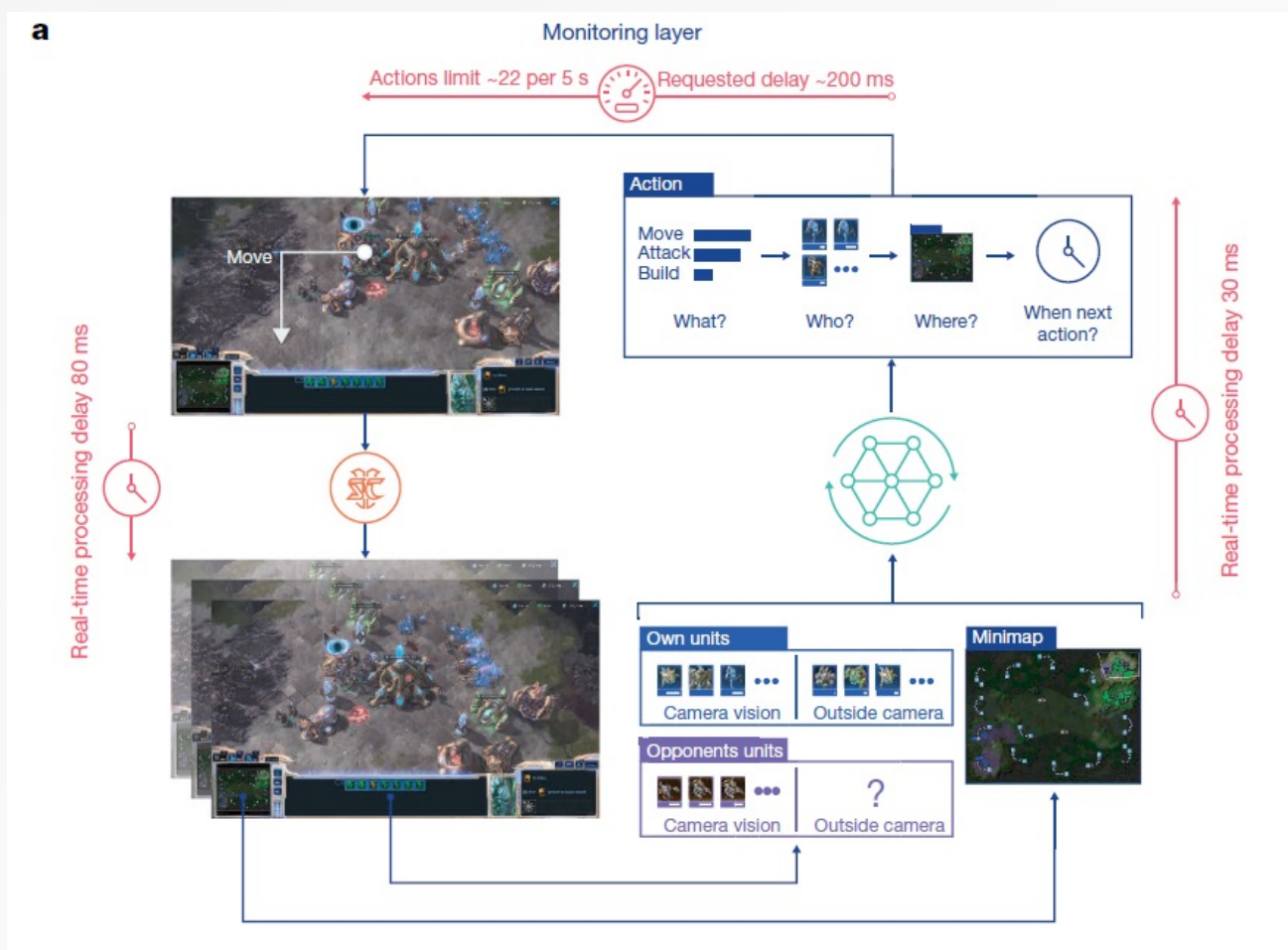
Outline

- Introduction
- Related works
- **Algorithm details**
- Experiments and Results
- Conclusion

CASIA

Supervised learning

Overall framework:



Supervised learning

Input

- `prev_state` - The previous LSTM state.
- `entity_list` - The list of entities within the game.
- `map` - Seven layers that describe the game map.
- `scalar_features` - Player data and game statistic information (see interface section for details), as well as build orders

Supervised learning

Input

- `opponent_observations` - The opponent's observations (which has a similar structure of ``entity_list``, ``map``, and ``scalar_features``). Only used for baselines, not for inference during play
- `cumulative_score` - Various score metrics tracked by the game, and only used for baselines, not for inference during play.

Supervised learning

Output

- The probability distribution of
- What
- Who
- Where
- The reason is to save action space

Supervised learning

Output Example



Human Actions

IDLE

Left_Click_Hold (p1)



Press



Release (p2)



Left_Click (p3)



IDLE

Agent Actions

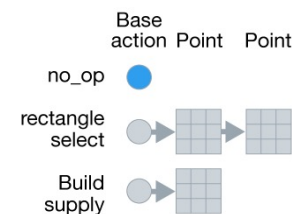
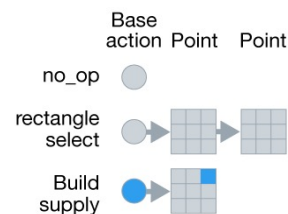
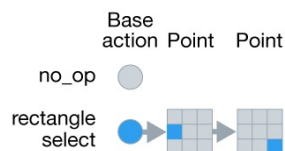
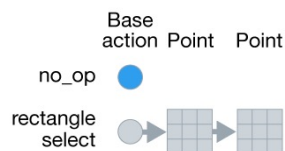
no_op

select_rect(p1, p2)

build_supply(p3)

no_op

Available Actions



Supervised learning

Loss Function

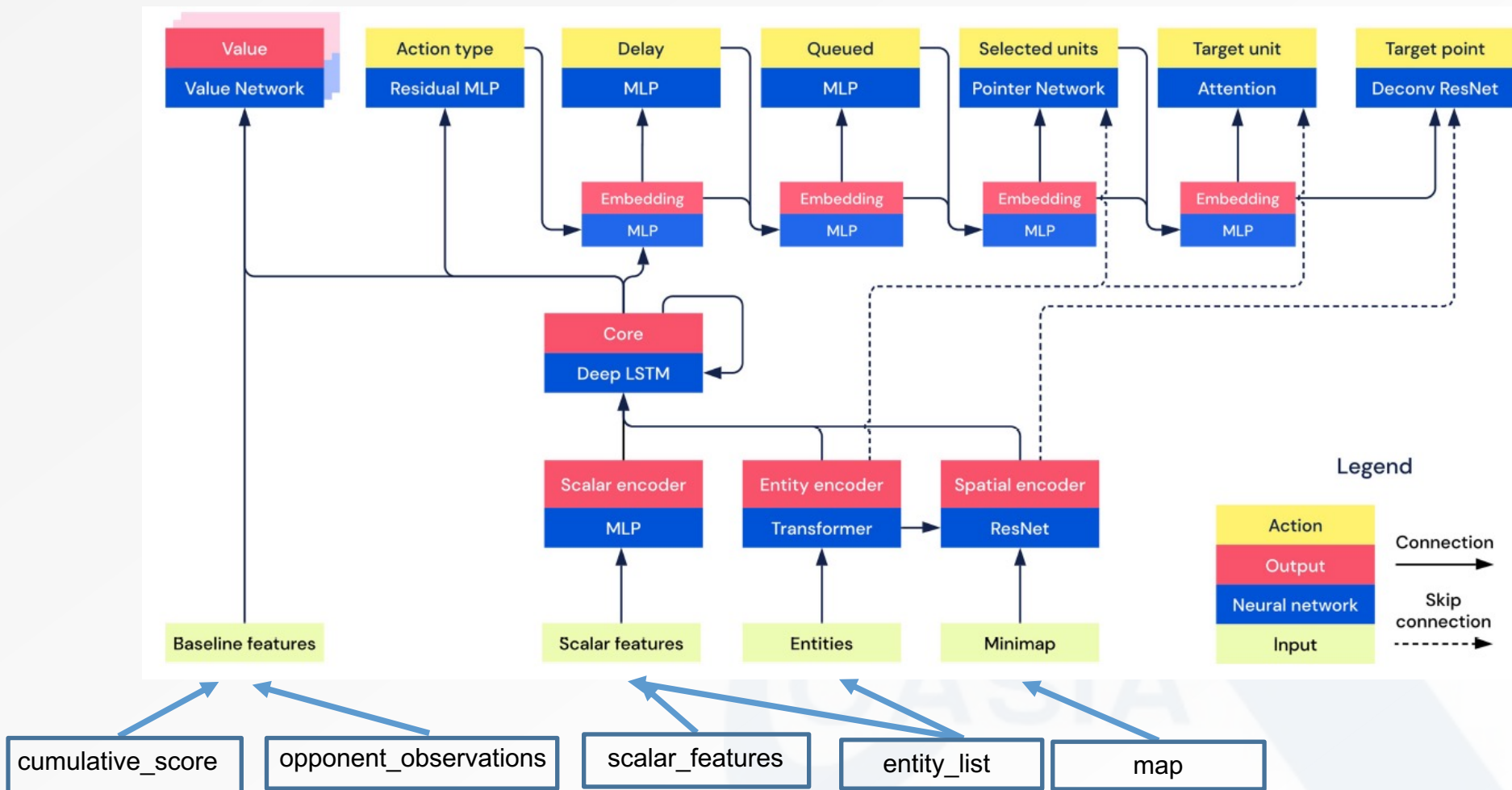
- $D_{KL}(p||q) = \sum_{i=1}^N p(x_i) \log \frac{p(x_i)}{q(x_i)}$
- $D_{KL}(p_{what}||q_{what}) + D_{KL}(p_{who}||q_{who}) + D_{KL}(p_{where}||q_{where}) + \lambda L_2$

Optimizer

- Adam

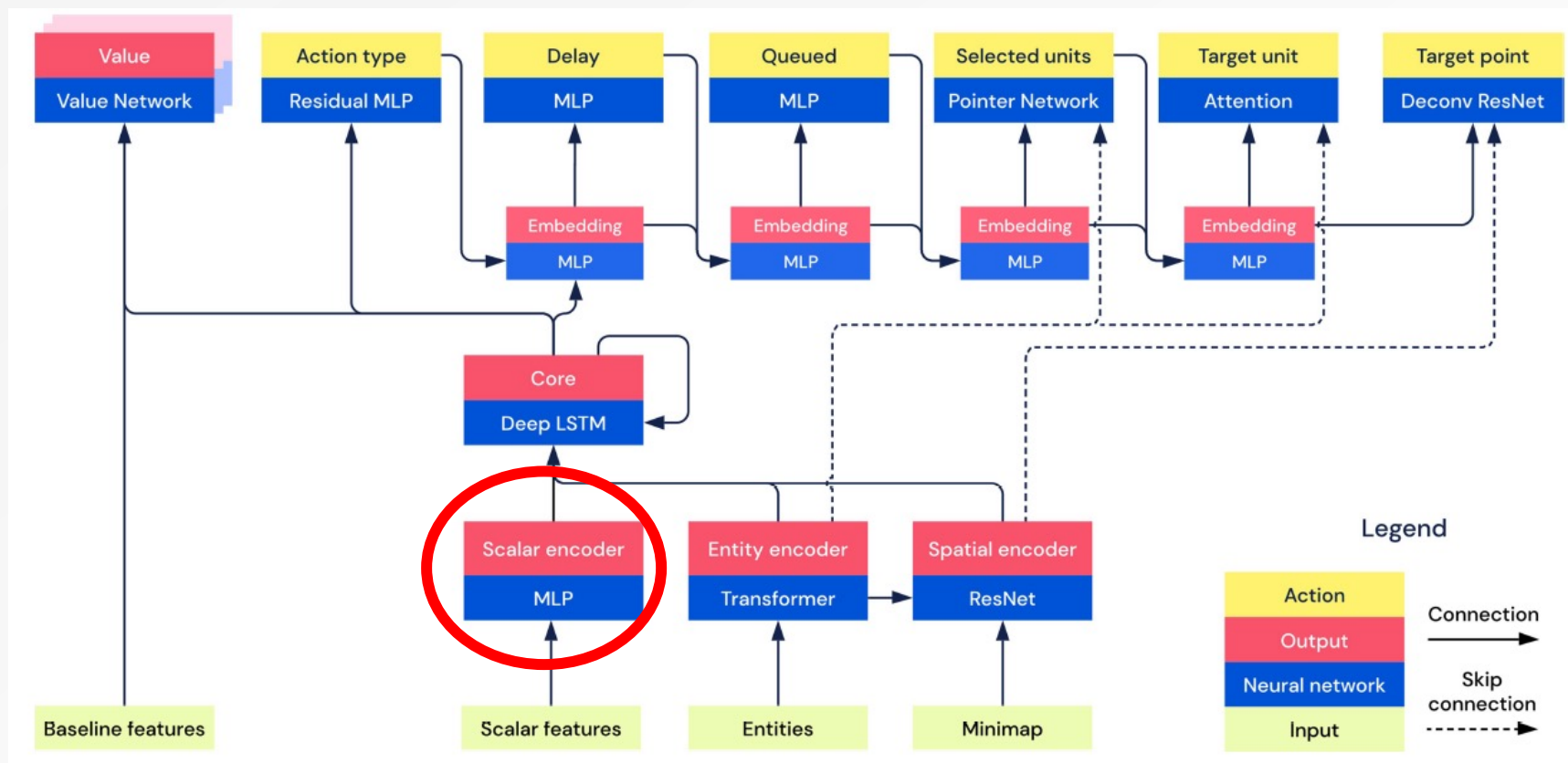
Supervised learning

Network Architecture



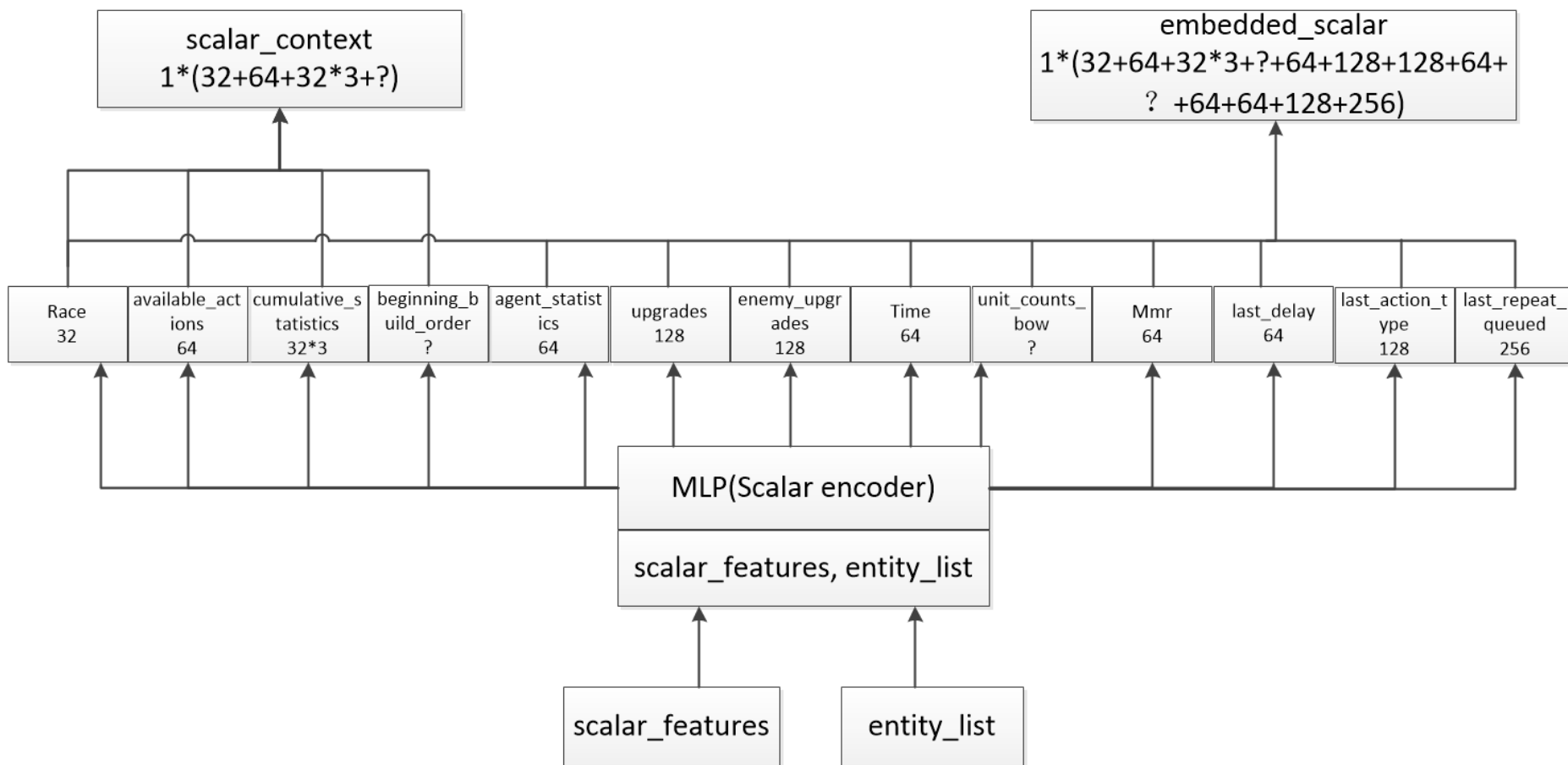
Supervised learning

Scalar encoder



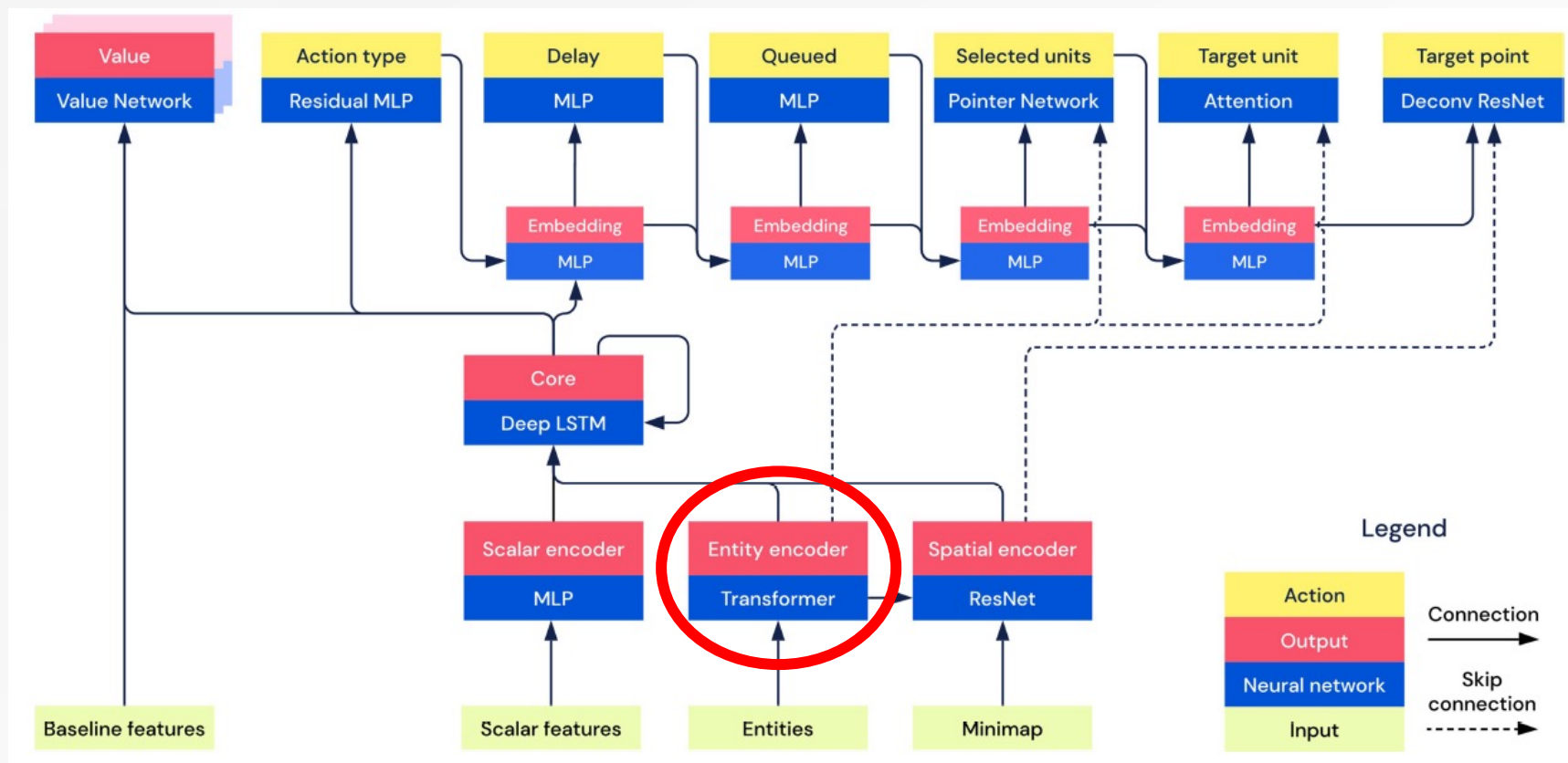
Supervised learning

Scalar encoder



Supervised learning

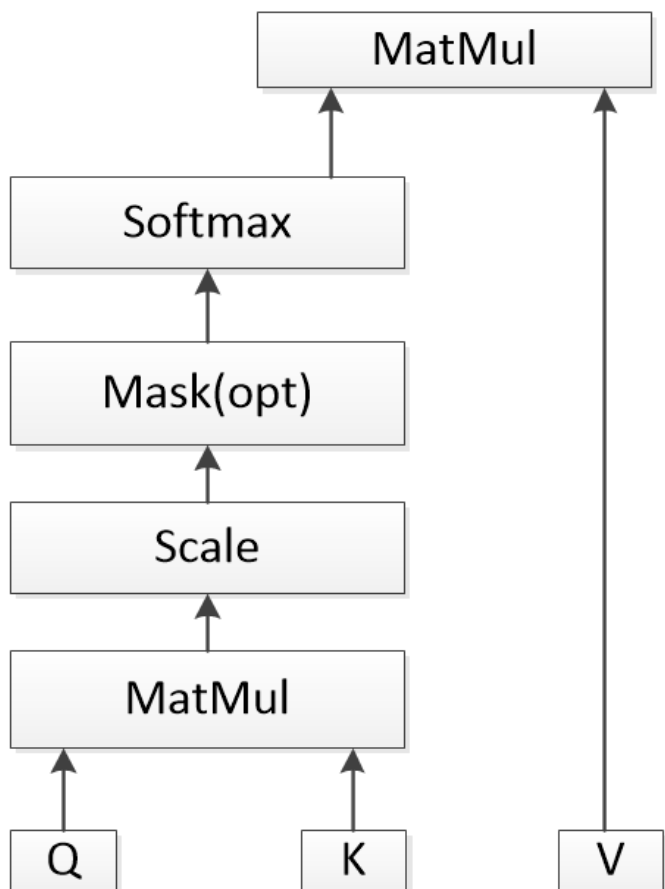
Entity encoder



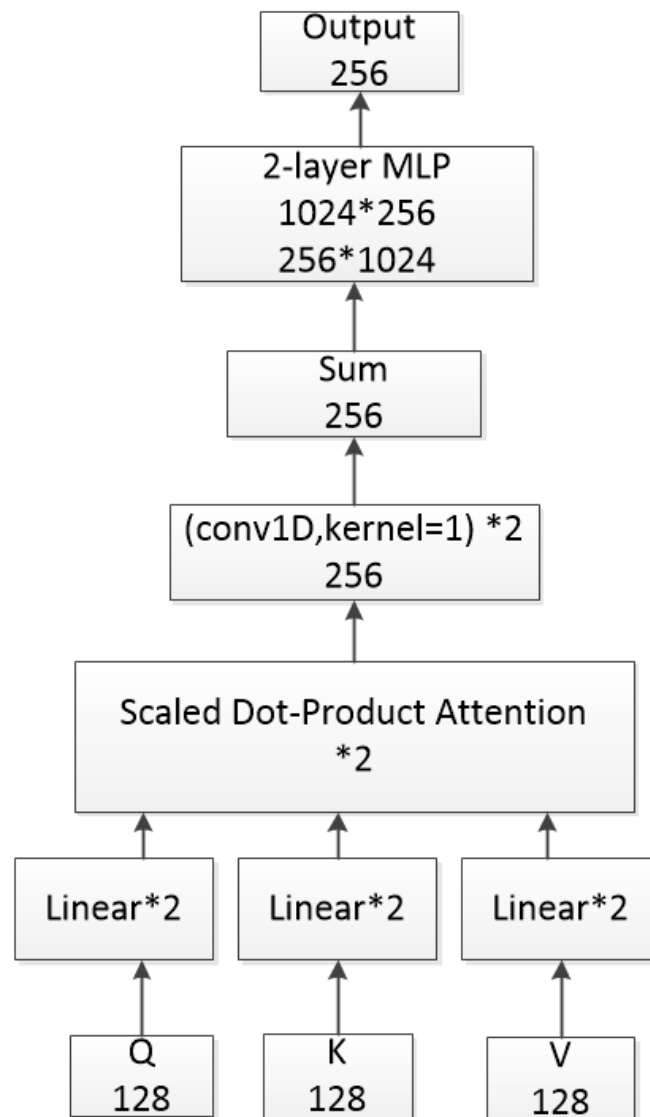
Supervised learning

Entity encoder

Scaled Dot-Product Attention

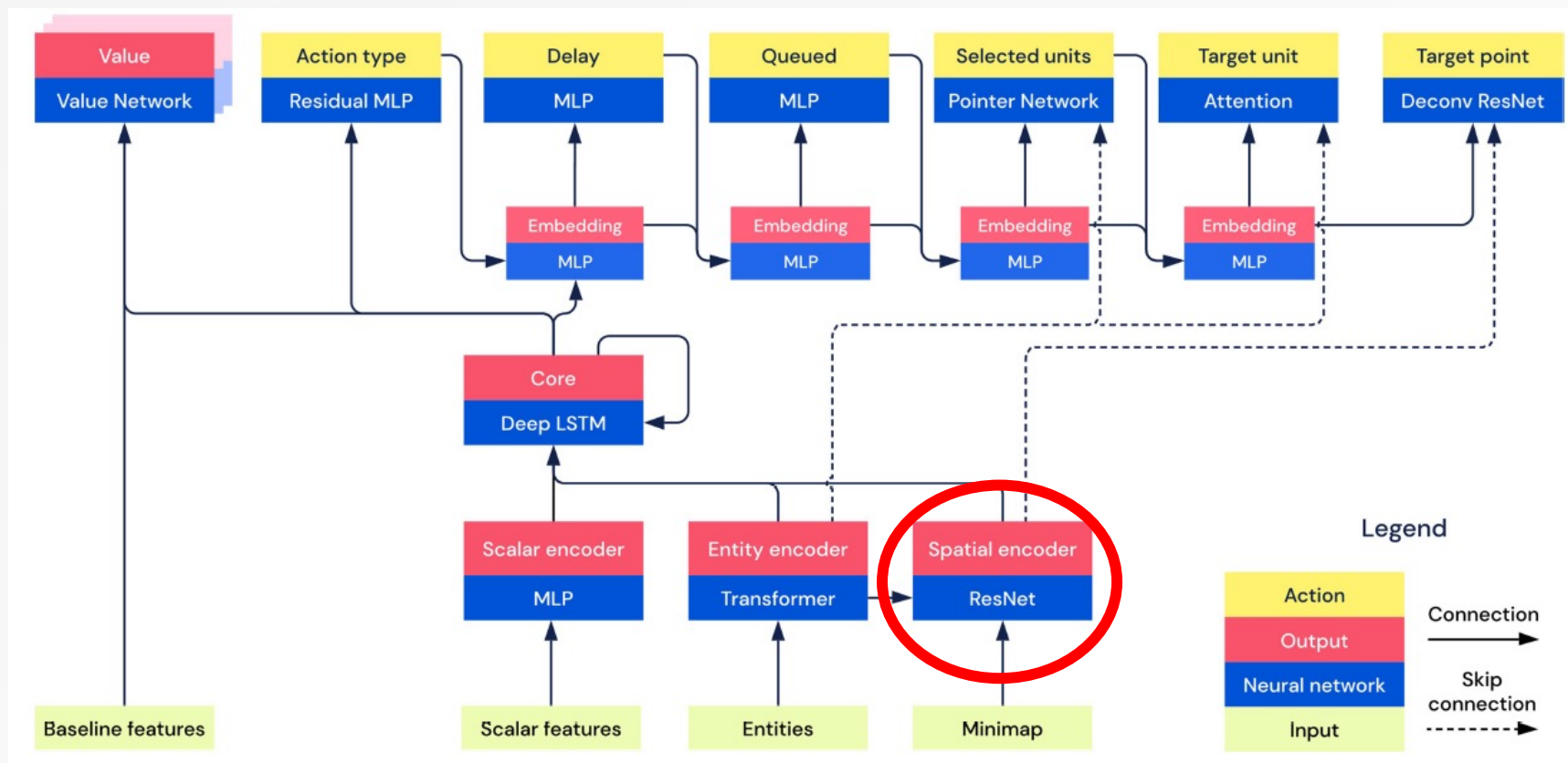


2-Head Attention



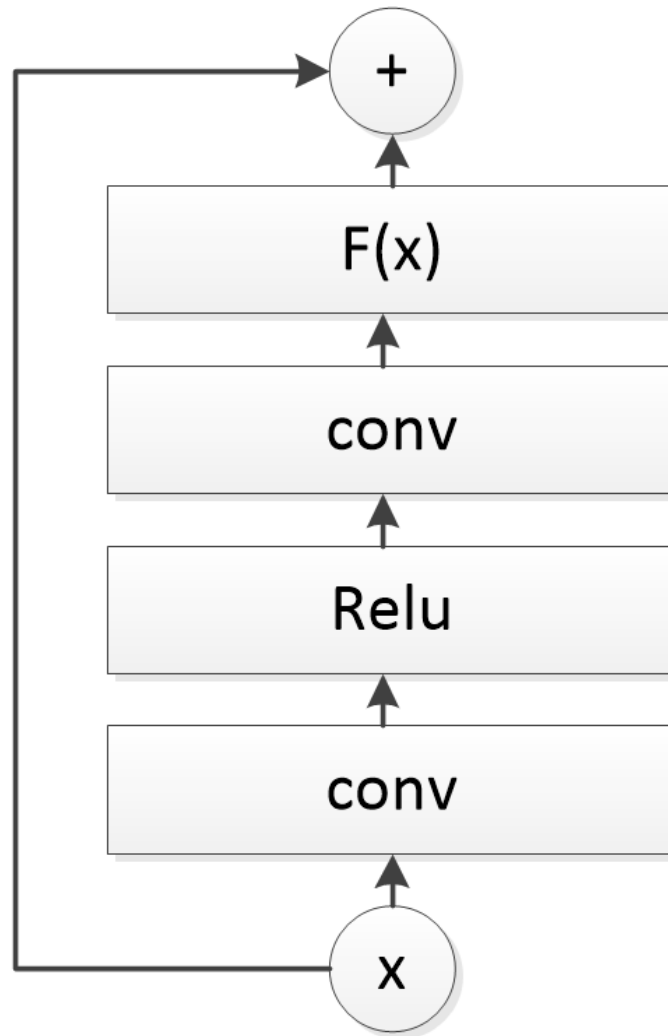
Supervised learning

Spatial encoder

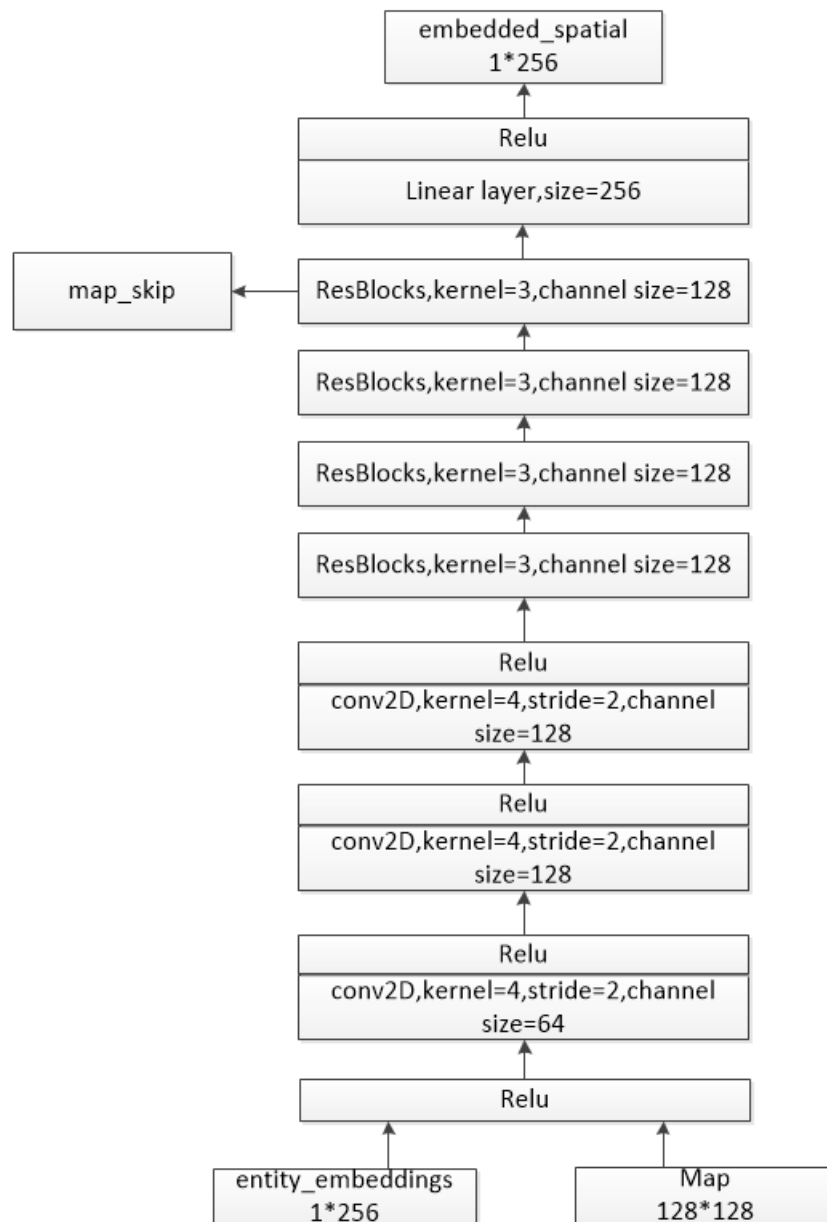


Supervised learning

Spatial encoder

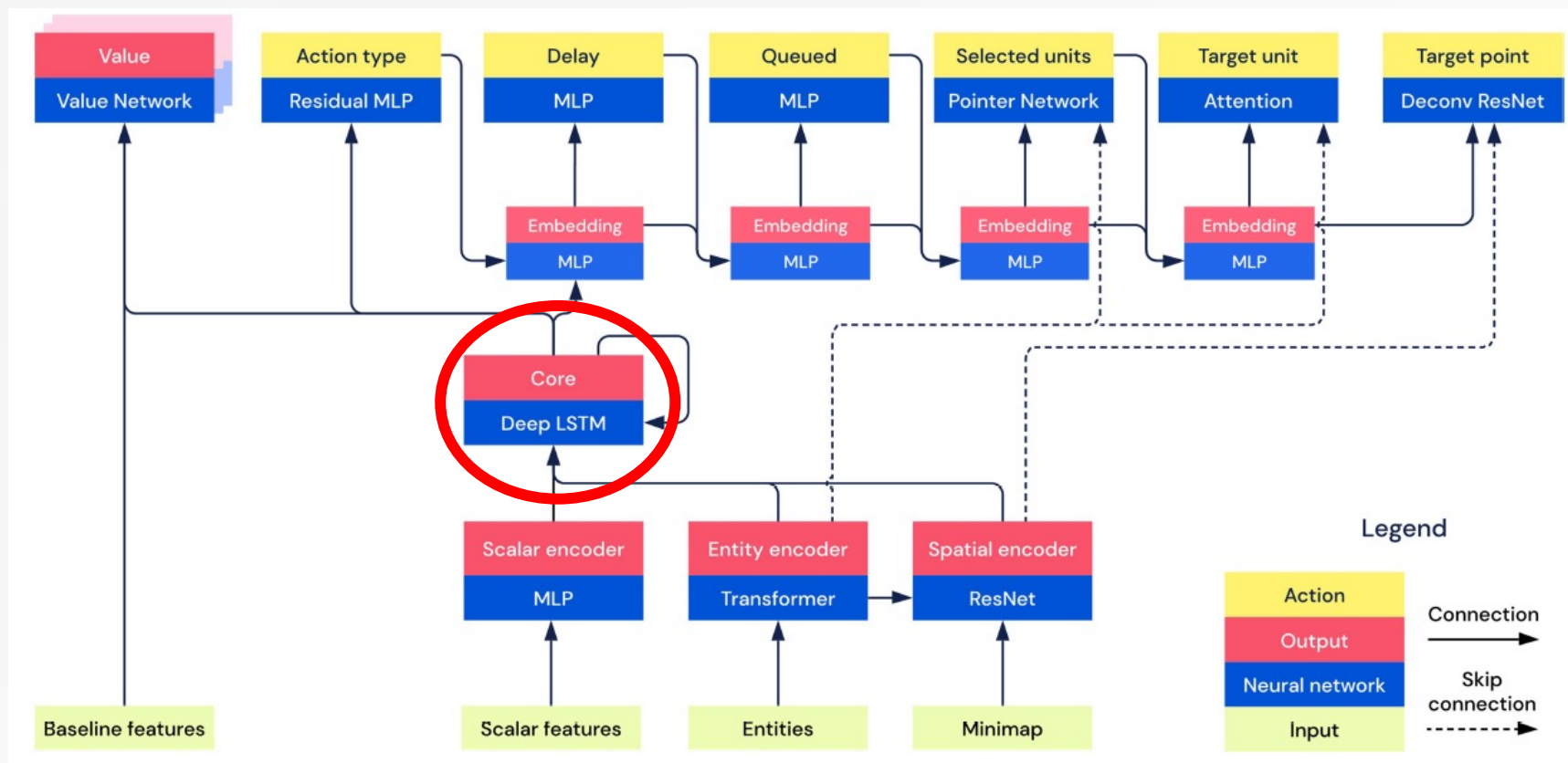


Supervised learning



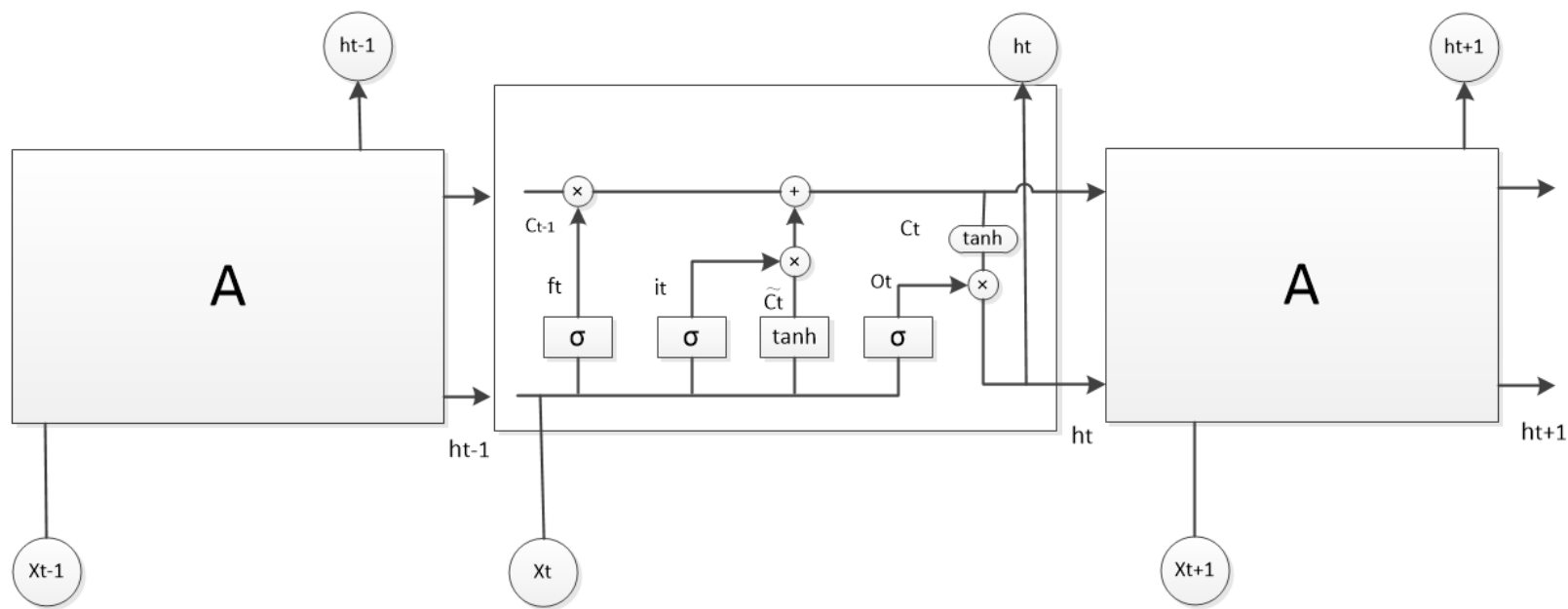
Supervised learning

Core

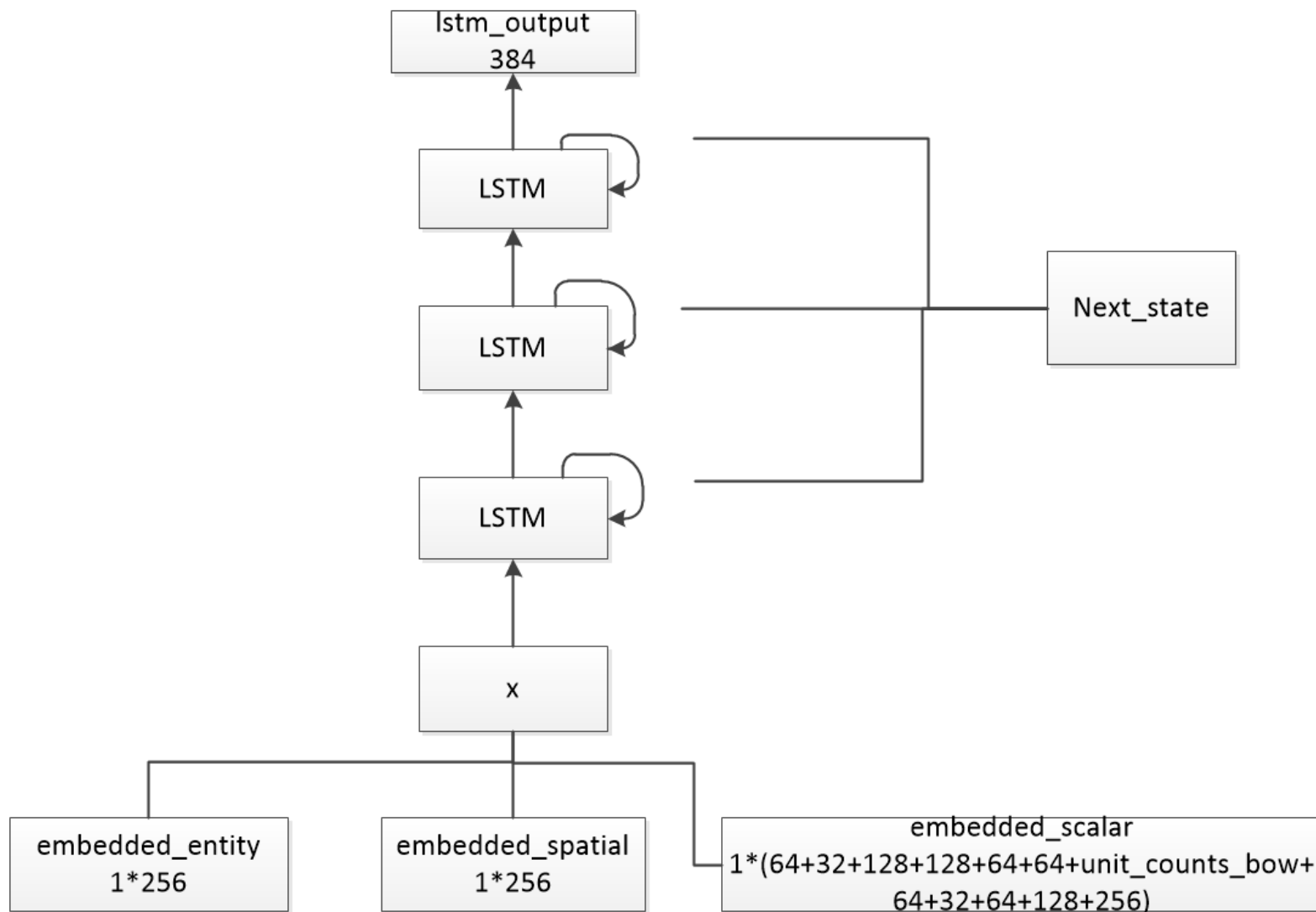


Supervised learning

Core

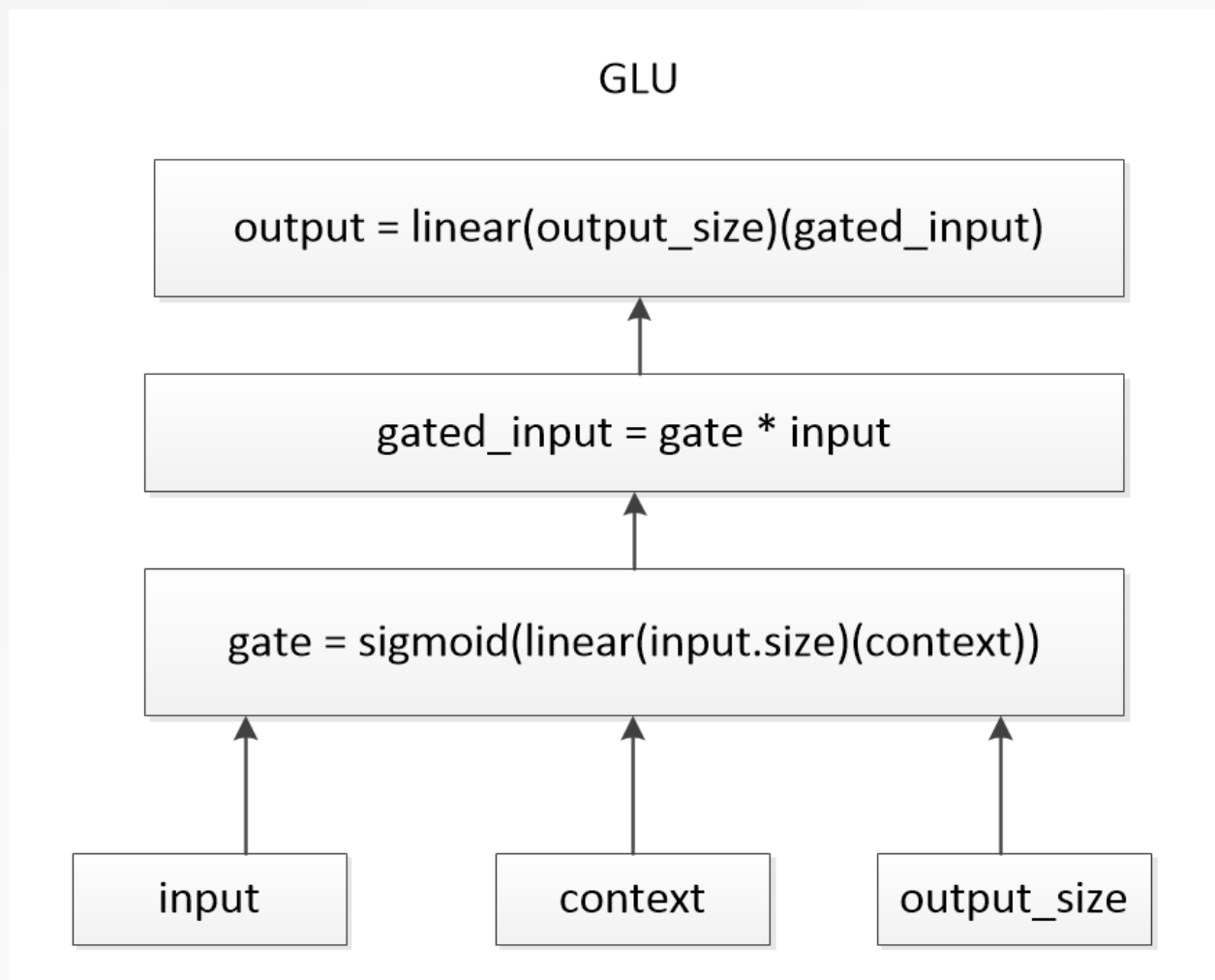


Supervised learning



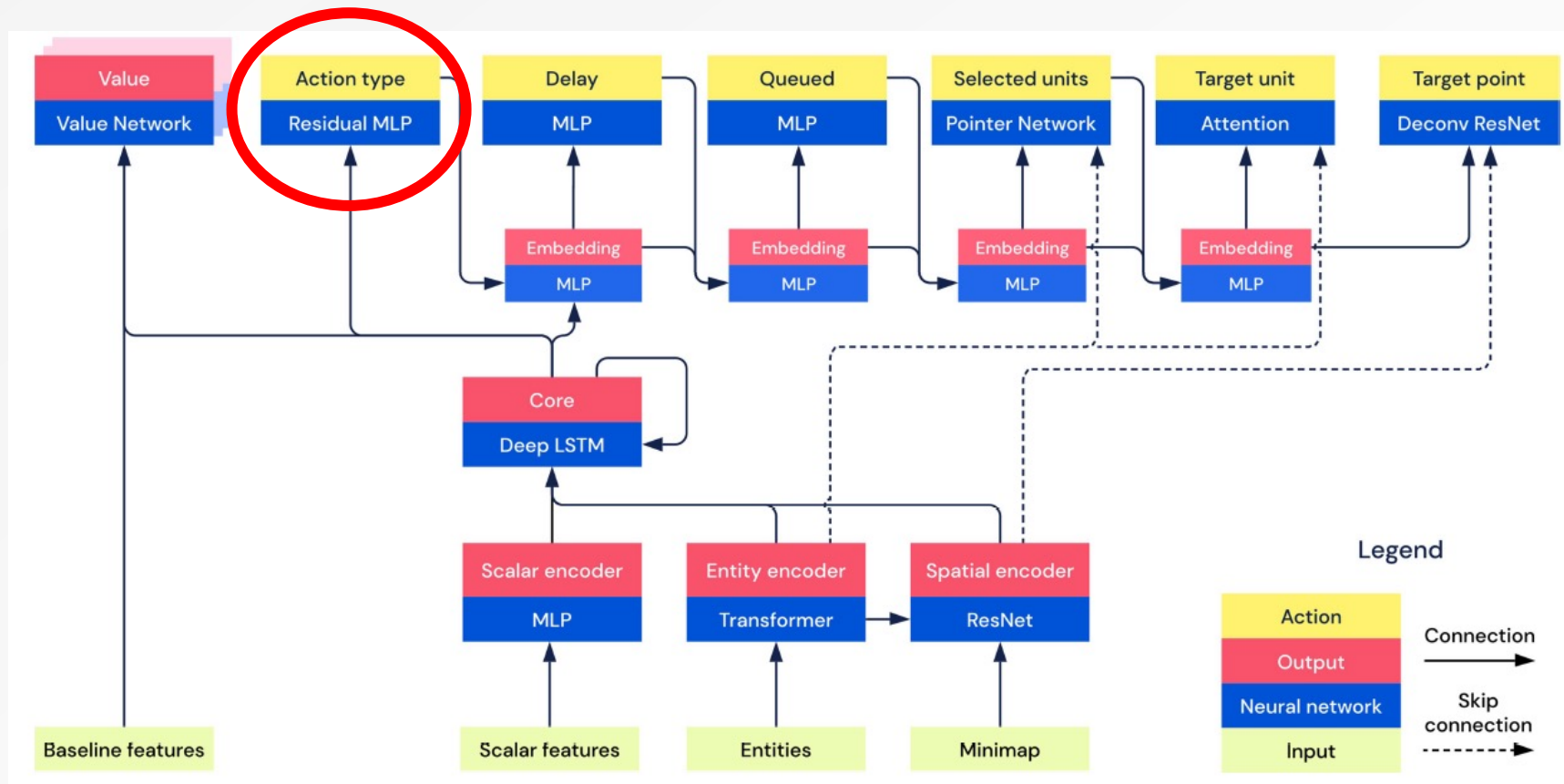
Supervised learning

GLU



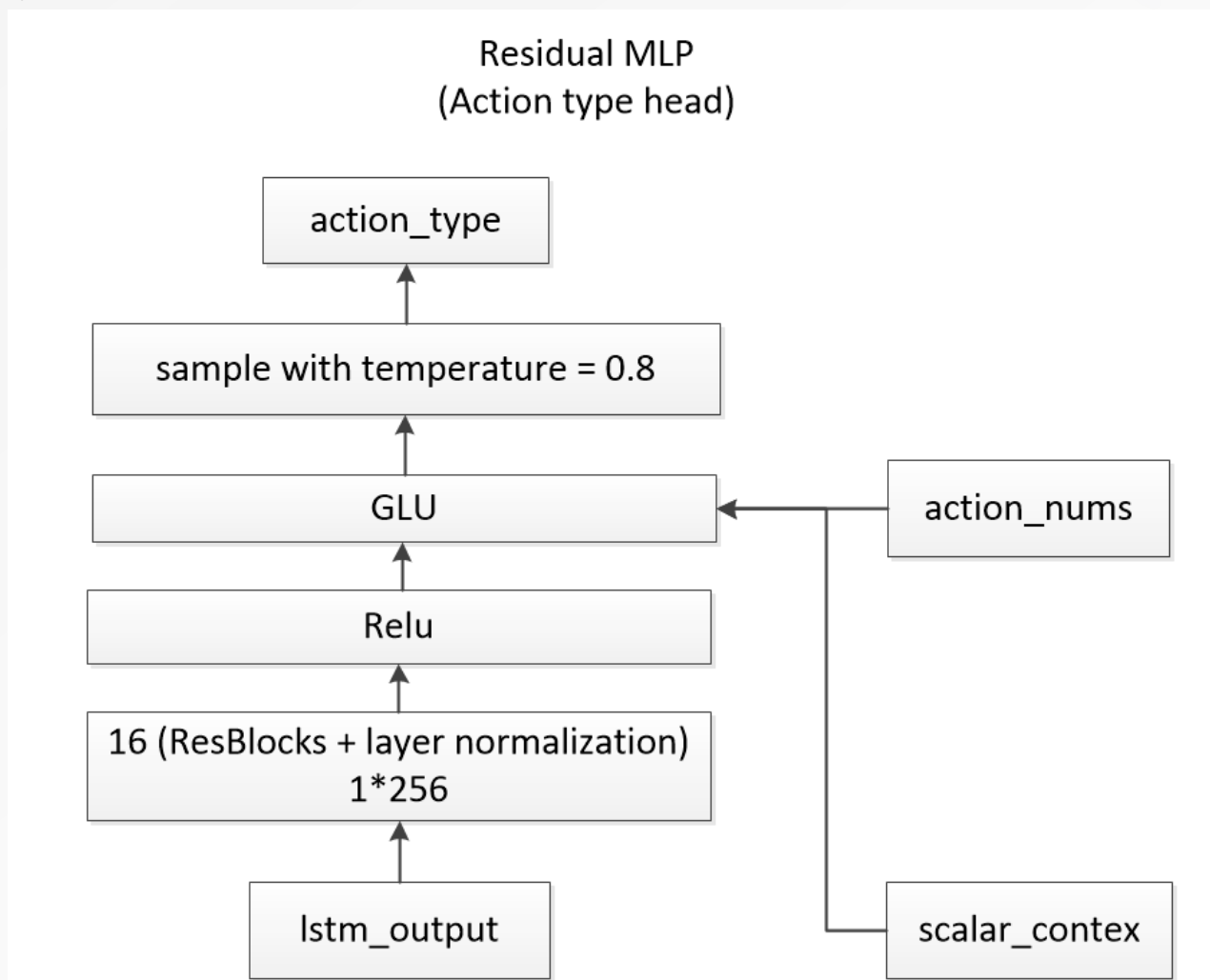
Supervised learning

Action type head

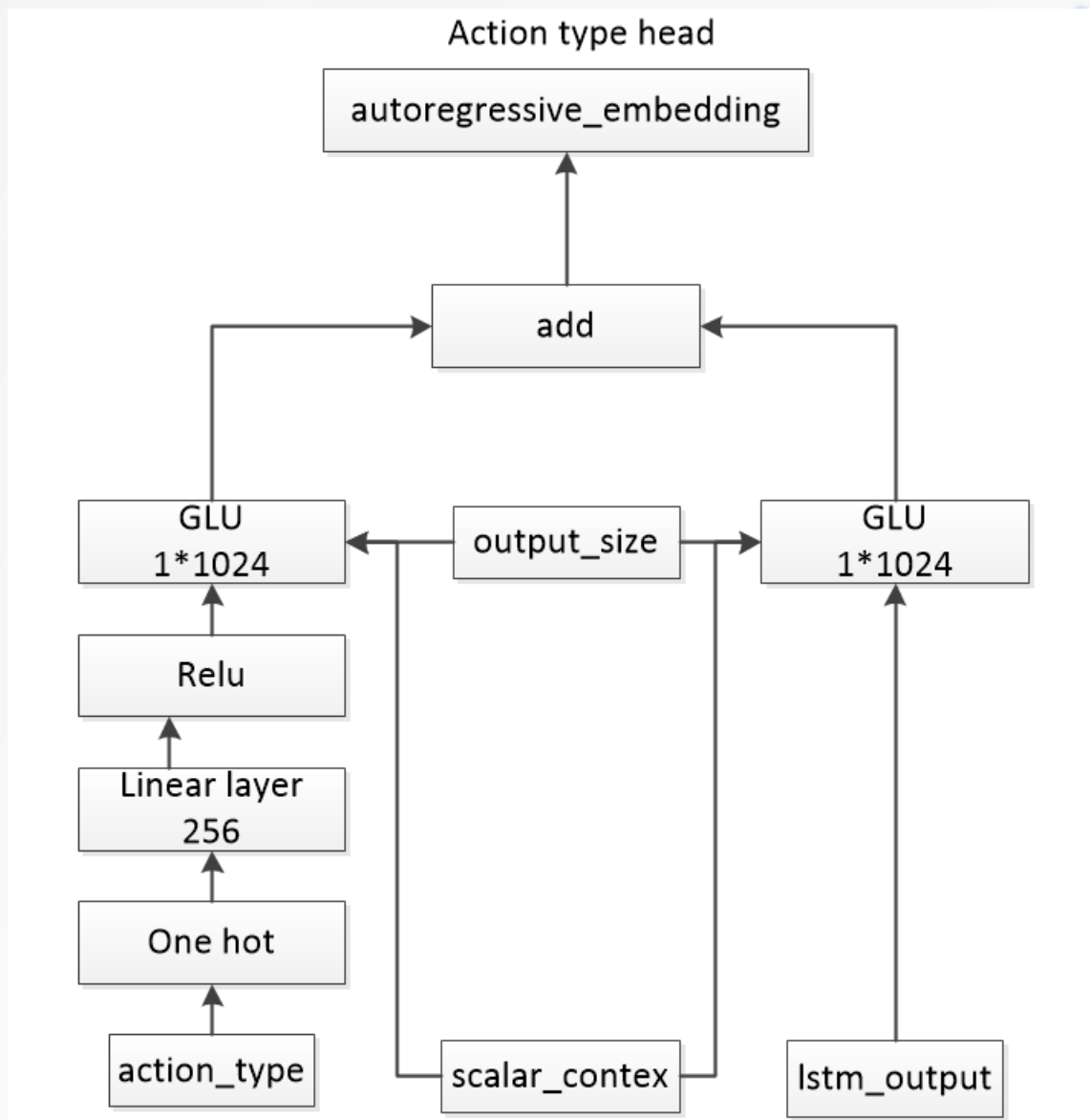


Supervised learning

Action type head

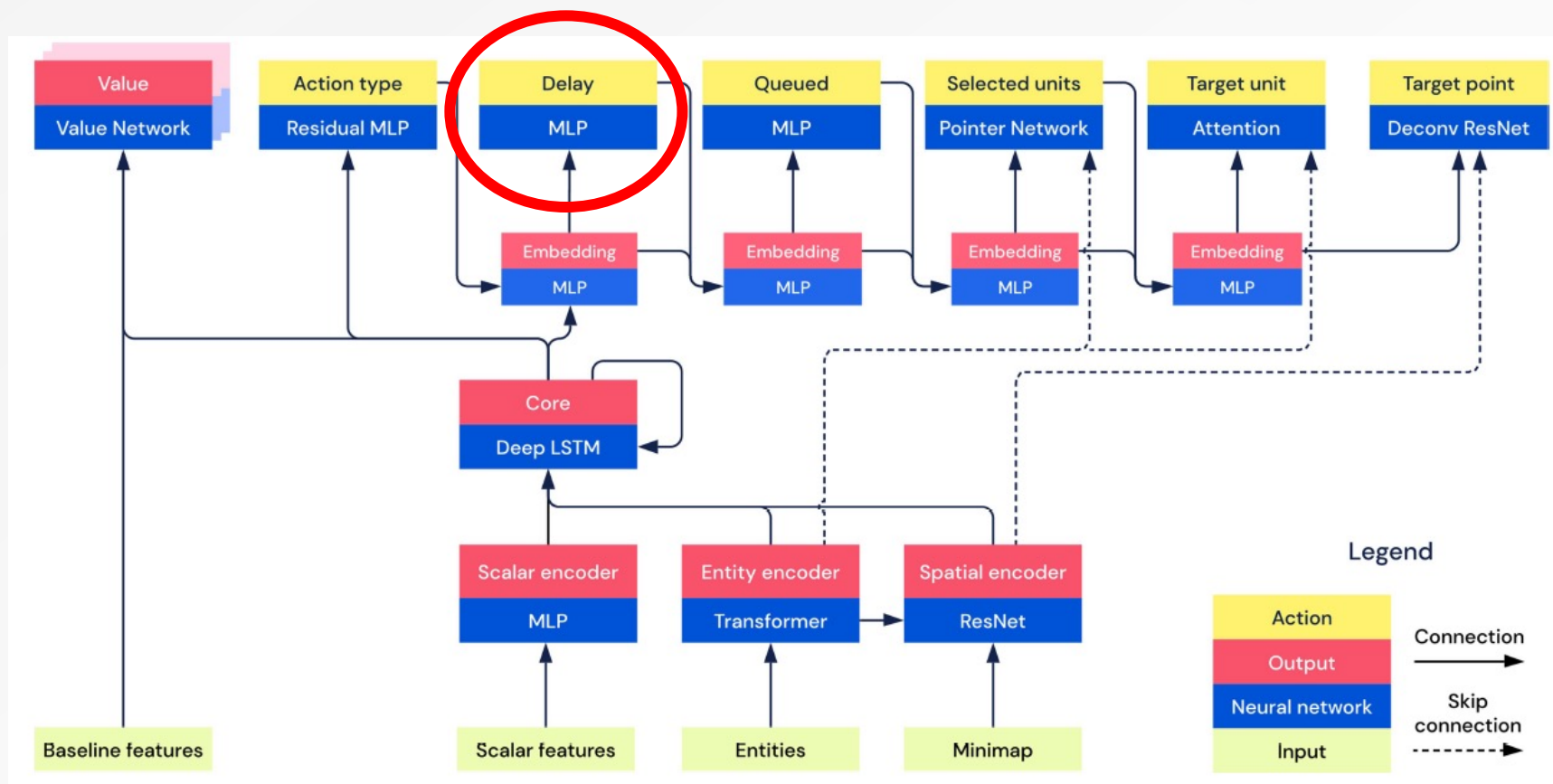


Supervised learning



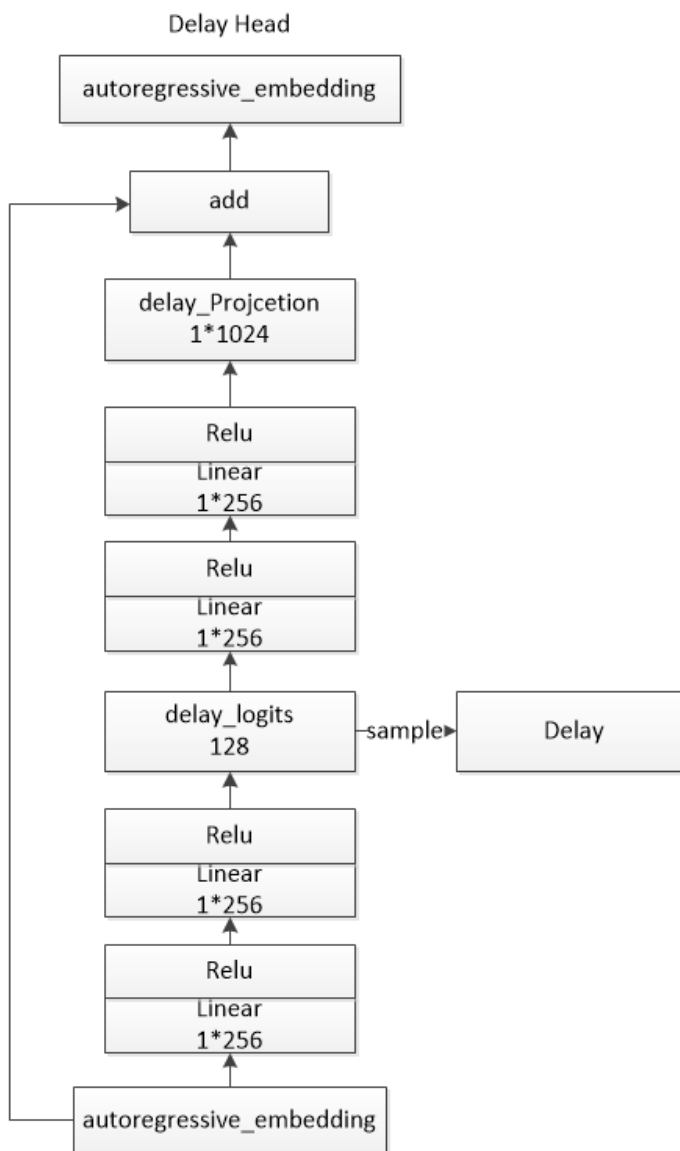
Supervised learning

Delay Head



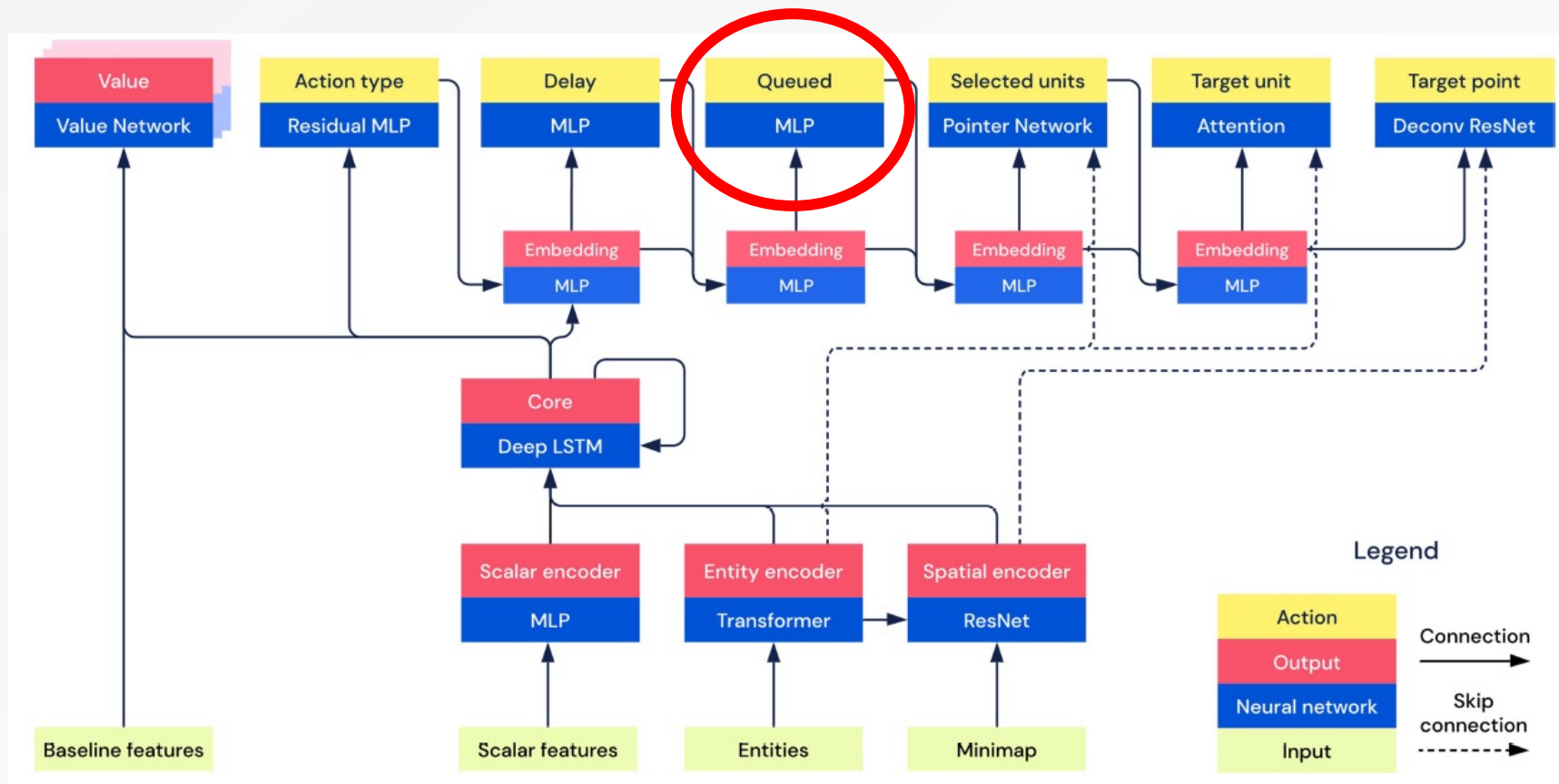
Supervised learning

Delay Head



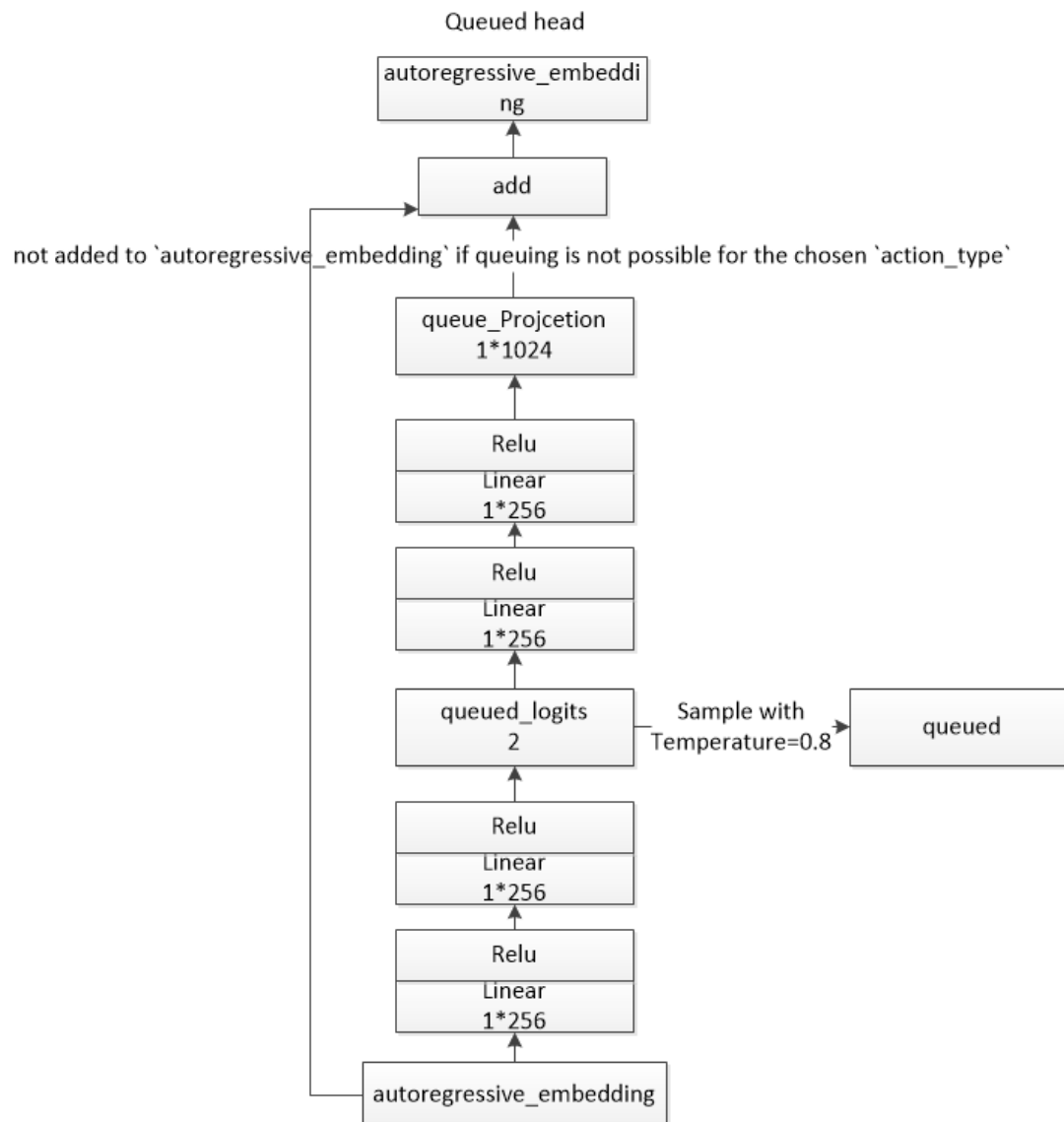
Supervised learning

Queued head



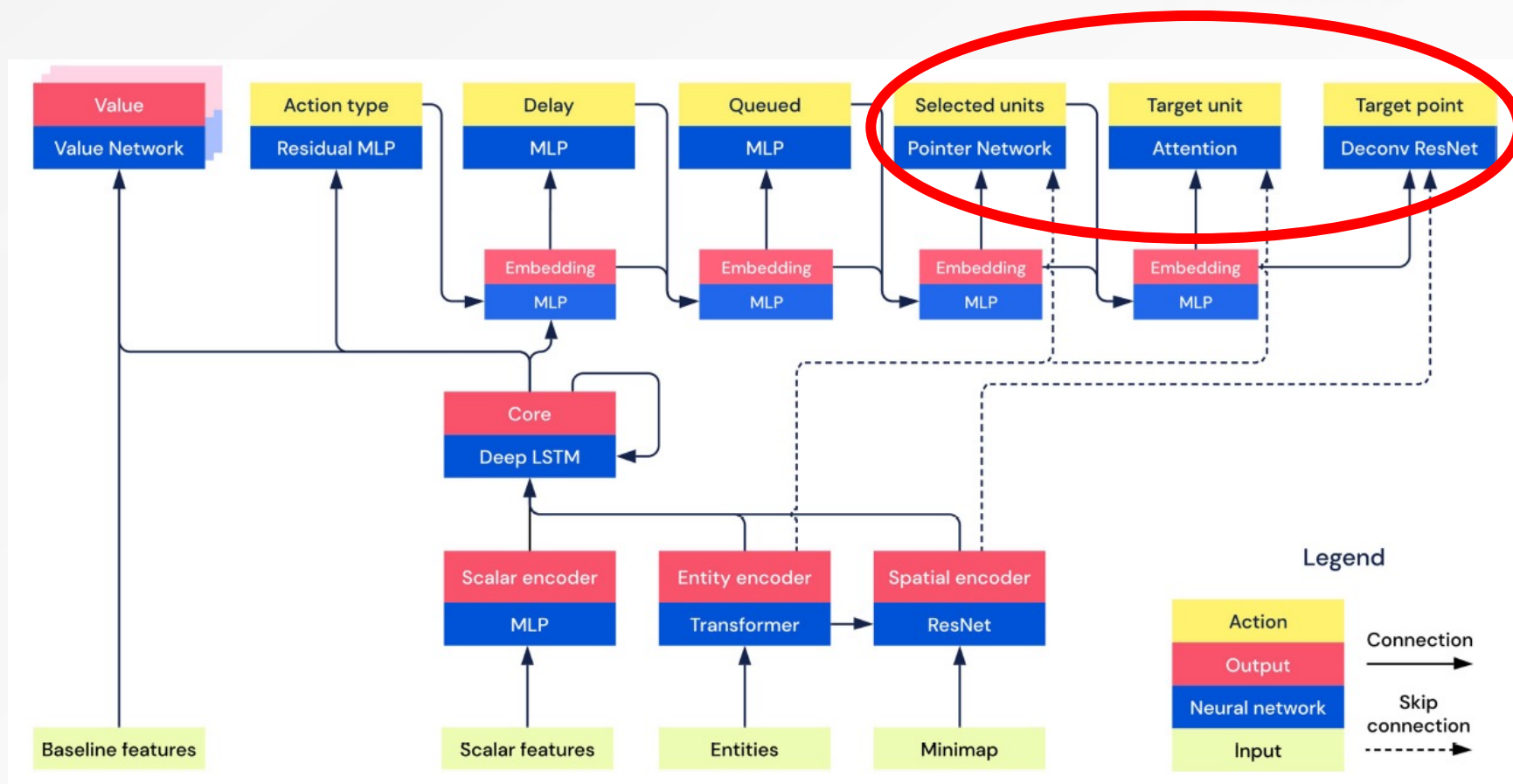
Supervised learning

Queued head



Supervised learning

Others

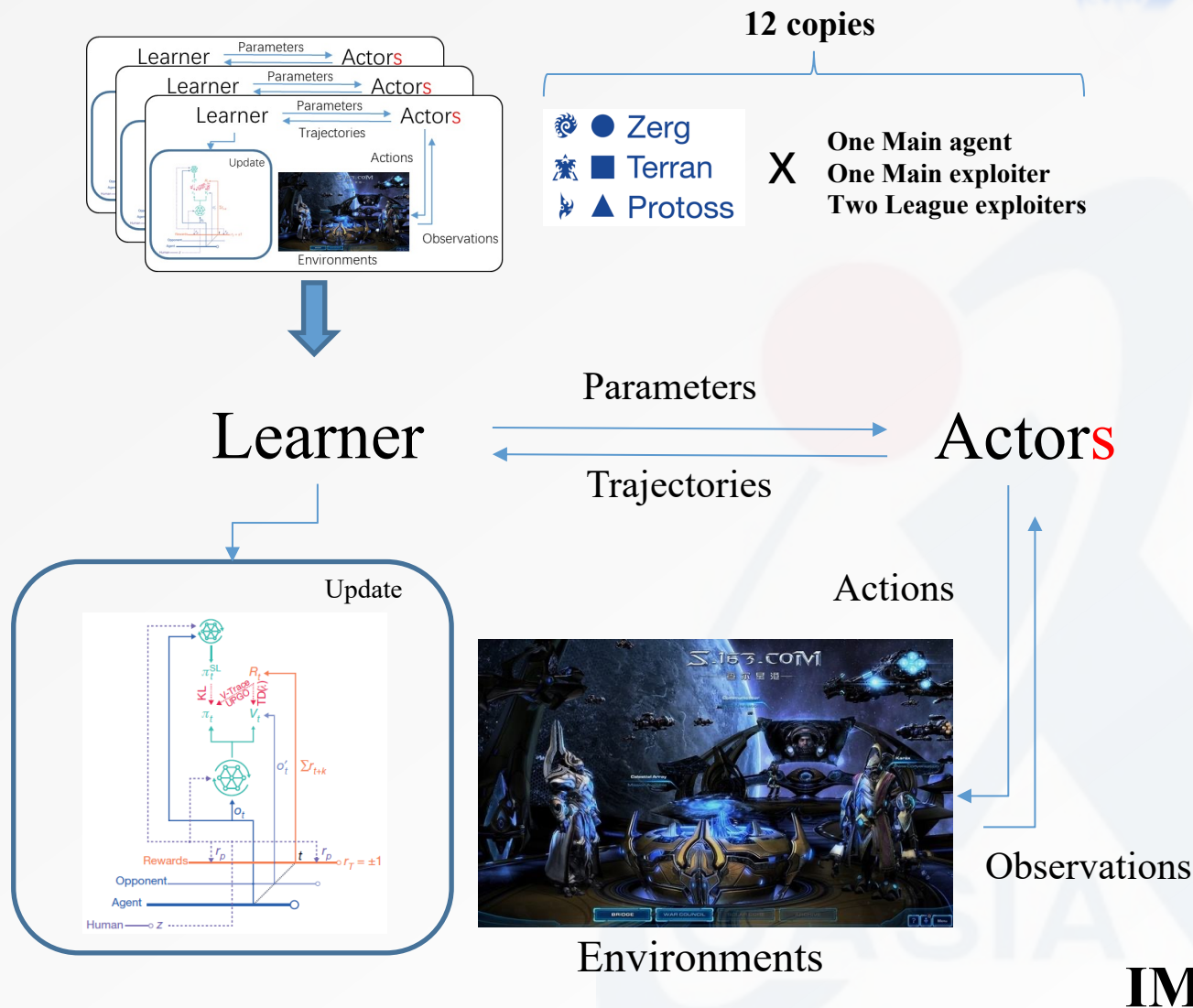


Reinforcement learning

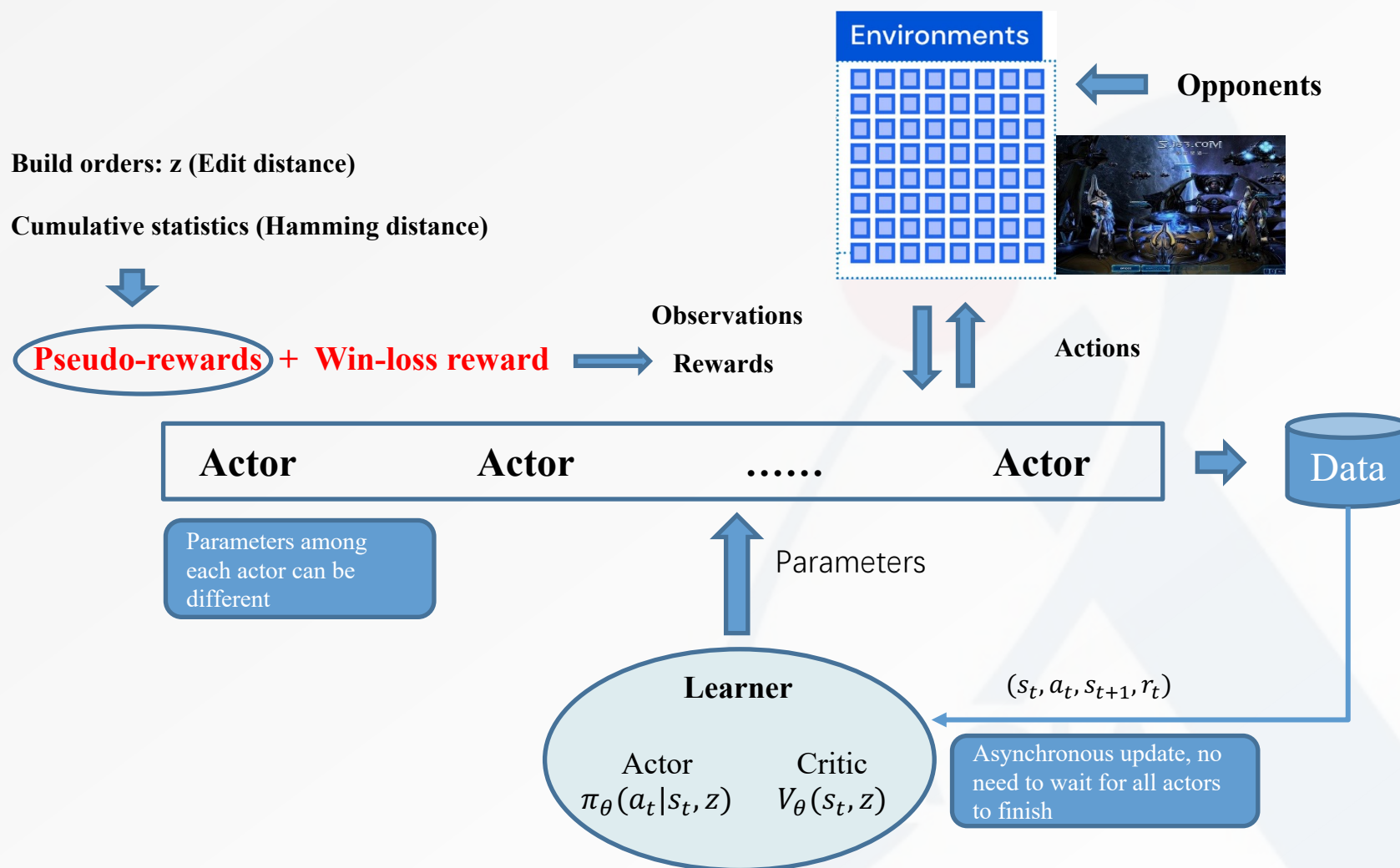
Challenges when views StarCraft as a reinforcement learning problem:

- Exploration is difficult: domain complexity and reward sparsity
- Policies need to be capable of executing diverse strategies throughout training
- Off-policy learning is difficult: large time horizons and the complex action space

Reinforcement learning



Reinforcement learning



Reinforcement learning

Learner

Actor Critic
 $\pi_{\theta}(a_t|s_t, z)$ $V_{\theta}(s_t, z)$

V-trace loss : $\rho_s(r_t + \gamma v_{t+1} - V_{\theta}(s_t, z)) \nabla_{\theta} \log \pi_{\theta}(a_t|s_t, z)$

$$v_t \stackrel{\text{def}}{=} V(s_t) + \sum_{t=s}^{s+n-1} \gamma^{t-s} \left(\prod_{i=s}^{t-1} c_i \right) \delta_t V$$

$$\delta_t V \stackrel{\text{def}}{=} \rho_t(r_t + \gamma V(s_{t+1}) - V(s_t))$$

$$\rho_t = \min(\bar{\rho}, \frac{\pi(a_t|s_t)}{\mu(a_t|s_t)}), c_i = \min(\bar{c}, \frac{\pi(a_i|s_i)}{\mu(a_i|x_i)})$$

UPGO loss : $\rho_t (G_t^U - V_{\theta}(s_t, z)) \nabla_{\theta} \log \pi_{\theta}(a_t|s_t, z)$

$$G_t^U = \begin{cases} r_t + G_{t+1}^U, & \text{if } Q(s_{t+1}, a_{t+1}, z) \geq V_{\theta}(s_{t+1}, z) \\ r_t + V_{\theta}(s_{t+1}, z), & \text{otherwise} \end{cases}$$

$$\rho_t = \min\left(\frac{\pi_{\theta}(a_t|s_t, z)}{\pi_{\theta'}(a_t|s_t, z)}, 1\right)$$

Human policy loss (KL divergence) : $D_{kl}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$

Entropy regularization loss : $-\nabla_{\theta} \sum_{a_t} \pi_{\theta}(s_t|s_t, z) \log \pi_{\theta}(a_t|s_t, z)$

TD(λ): $V(s_t) \leftarrow V(s_t) + \alpha(G_t^{\lambda} - V(s_t))$

$$G_t^{\lambda} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(s_{t+n})$$

Reinforcement learning

$$TD(\lambda): V(S_t) \leftarrow V(S_t) + \alpha(G_t^\lambda - V(S_t))$$

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

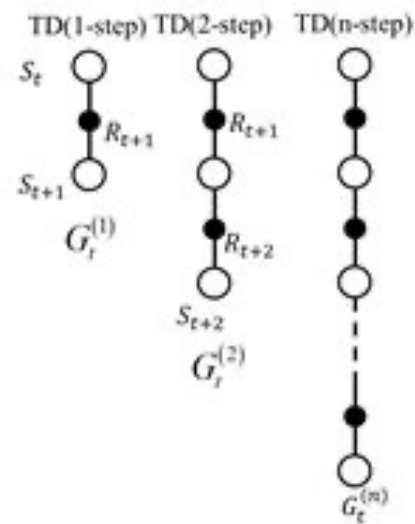
$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$



$$n = 1 \text{ (TD)} \quad G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$$

$$n = 2 \quad G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$

$$n = \infty \text{ (MC)} \quad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

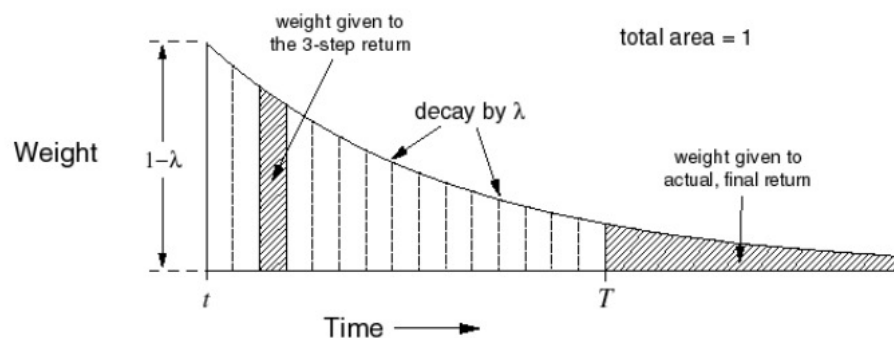
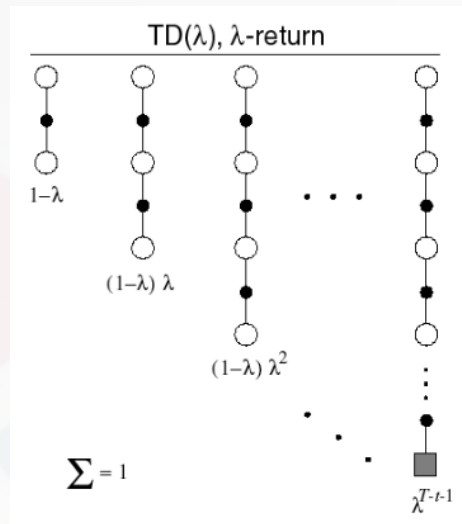


Reinforcement learning

$$TD(\lambda): V(S_t) \leftarrow V(S_t) + \alpha(G_t^\lambda - V(S_t))$$

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$



$$\lambda = 0, G_t^\lambda = G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}) \quad TD$$

$$\lambda = 1, G_t^\lambda = G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T \quad MC$$

Reinforcement learning

V-trace loss : $\rho_s(r_t + \gamma v_{t+1} - V_\theta(s_t, z)) \nabla_\theta \log \pi_\theta(a_t | s_t, z)$

$$v_t \stackrel{\text{def}}{=} V(s_t) + \sum_{t=s}^{s+n-1} \gamma^{t-s} \left(\prod_{i=s}^{t-1} c_i \right) \delta_t V$$

$$\delta_t V \stackrel{\text{def}}{=} \rho_t(r_t + \gamma V(s_{t+1}) - V(s_t))$$

$$\rho_t = \min(\bar{\rho}, \frac{\pi(a_t | s_t)}{\mu(a_t | s_t)}), c_i = \min(\bar{c}, \frac{\pi(a_i | s_i)}{\mu(a_i | x_i)})$$

UPGO loss : $\rho_t (G_t^U - V_\theta(s_t, z)) \nabla_\theta \log \pi_\theta(a_t | s_t, z)$

$$G_t^U = \begin{cases} r_t + G_{t+1}^U, & \text{if } Q(s_{t+1}, a_{t+1}, z) \geq V_\theta(s_{t+1}, z) \\ r_t + V_\theta(s_{t+1}, z), & \text{otherwise} \end{cases}$$

$$\rho_t = \min(\frac{\pi_\theta(a_t | s_t, z)}{\pi_{\theta'}(a_t | s_t, z)}, 1)$$

Human policy loss (KL divergence) : $D_{kl}(P || Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$

Entropy regularization loss : $-\nabla_\theta \sum_{a_t} \pi_\theta(s_t | s_t, z) \log \pi_\theta(a_t | s_t, z)$

Improvement of PG

Policy Gradient:

$$\max_{\theta} U(\theta) = \max_{\theta} E(R(\tau)) = \max_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau)$$

$$\nabla_{\theta} U_{\theta} = \nabla_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau)$$

$$\Rightarrow (G_t - b) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t, z)$$

return

reduce the variance

Reinforcement learning

V-trace loss : $\rho_s(r_t + \gamma v_{t+1} - V_\theta(s_t, z)) \nabla_\theta \log \pi_\theta(a_t | s_t, z)$

$$v_t \stackrel{\text{def}}{=} V(s_t) + \sum_{t=s}^{s+n-1} \gamma^{t-s} \left(\prod_{i=s}^{t-1} c_i \right) \delta_t V$$

$$\delta_t V \stackrel{\text{def}}{=} \rho_t(r_t + \gamma V(s_{t+1}) - V(s_t))$$

$$\rho_t = \min(\bar{\rho}, \frac{\pi(a_t | s_t)}{\mu(a_t | s_t)}), c_i = \min(\bar{c}, \frac{\pi(a_i | s_i)}{\mu(a_i | x_i)})$$

UPGO loss : $\rho_t (G_t^U - V_\theta(s_t, z)) \nabla_\theta \log \pi_\theta(a_t | s_t, z)$

$$G_t^U = \begin{cases} r_t + G_{t+1}^U, & \text{if } Q(s_{t+1}, a_{t+1}, z) \geq V_\theta(s_{t+1}, z) \\ r_t + V_\theta(s_{t+1}, z), & \text{otherwise} \end{cases}$$

$$\rho_t = \min(\frac{\pi_\theta(a_t | s_t, z)}{\pi_{\theta'}(a_t | s_t, z)}, 1)$$

Human policy loss (KL divergence) : $D_{kl}(P || Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$

Entropy regularization loss : $-\nabla_\theta \sum_{a_t} \pi_\theta(s_t | s_t, z) \log \pi_\theta(a_t | s_t, z)$

Truncated importance sampling (IS) weights

Transfer on-policy method to off-policy method in order to improve the sample efficiency

Update the policy from partial trajectories with better-than-expected returns (self-imitation learning)

Reinforcement learning

V-trace loss : $\rho_s(r_t + \gamma v_{t+1} - V_\theta(s_t, z)) \nabla_\theta \log \pi_\theta(a_t | s_t, z)$

$$v_t \stackrel{\text{def}}{=} V(s_t) + \sum_{t=s}^{s+n-1} \gamma^{t-s} \left(\prod_{i=s}^{t-1} c_i \right) \delta_t V$$

$$\delta_t V \stackrel{\text{def}}{=} \rho_t(r_t + \gamma V(s_{t+1}) - V(s_t))$$

$$\rho_t = \min(\bar{\rho}, \frac{\pi(a_t | s_t)}{\mu(a_t | s_t)}), c_i = \min(\bar{c}, \frac{\pi(a_i | s_i)}{\mu(a_i | x_i)})$$

UPGO loss : $\rho_t (G_t^U - V_\theta(s_t, z)) \nabla_\theta \log \pi_\theta(a_t | s_t, z)$

$$G_t^U = \begin{cases} r_t + G_{t+1}^U, & \text{if } Q(s_{t+1}, a_{t+1}, z) \geq V_\theta(s_{t+1}, z) \\ r_t + V_\theta(s_{t+1}, z), & \text{otherwise} \end{cases}$$

$$\rho_t = \min(\frac{\pi_\theta(a_t | s_t, z)}{\pi_{\theta'}(a_t | s_t, z)}, 1)$$

Human policy loss (KL divergence) : $D_{kl}(P || Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$

Learning from human data

Entropy regularization loss : $-\nabla_\theta \sum_{a_t} \pi_\theta(s_t | s_t, z) \log \pi_\theta(a_t | s_t, z)$

Increase exploration

Multi-agent learning

- Designing **League Training**: avoid recycle + integrate strategies
- **Avoiding recycle**: Recycle: for example: A defeats B, B defeats C, C defeats A.

Algorithm 2 Self-play

input: agent v_1

for $t = 1, \dots, T$ do

$v_{t+1} \leftarrow \text{oracle}(v_t, \phi_{v_t}(\bullet))$

end for

output: v_{T+1}

Return a new agent by using
gradients, reinforcement learning
or evolutionary algorithms.

May chase cycles.

Multi-agent learning

- FSP: Fictitious self-play:

Algorithm 2 General Fictitious Self-Play

function FICTITIOUSSELFPLAY(Γ, n, m)

Initialize completely mixed π_1

$\beta_2 \leftarrow \pi_1$

$j \leftarrow 2$

while within computational budget **do**

$\eta_j \leftarrow \text{MIXINGPARAMETER}(j)$

$\mathcal{D} \leftarrow \text{GENERATEDATA}(\pi_{j-1}, \beta_j, n, m, \eta_j)$

for each player $i \in \mathcal{N}$ **do**

$\mathcal{M}_{RL}^i \leftarrow \text{UPDATERLMEMORY}(\mathcal{M}_{RL}^i, \mathcal{D}^i)$

$\mathcal{M}_{SL}^i \leftarrow \text{UPDATESLMEMORY}(\mathcal{M}_{SL}^i, \mathcal{D}^i)$

$\beta_{j+1}^i \leftarrow \text{REINFORCEMENTLEARNING}(\mathcal{M}_{RL}^i)$

$\pi_j^i \leftarrow \text{SUPERVISEDLEARNING}(\mathcal{M}_{SL}^i)$

end for

$j \leftarrow j + 1$

end while

return π_{j-1}

end function

function GENERATEDATA(π, β, n, m, η)

$\sigma \leftarrow (1 - \eta)\pi + \eta\beta$

$\mathcal{D} \leftarrow n$ episodes $\{t_k\}_{1 \leq k \leq n}$, sampled from strategy profile σ

for each player $i \in \mathcal{N}$ **do**

$\mathcal{D}^i \leftarrow m$ episodes $\{t_k^i\}_{1 \leq k \leq m}$, sampled from strategy profile (β^i, σ^{-i})

$\mathcal{D}^i \leftarrow \mathcal{D}^i \cup \mathcal{D}$

end for

return $\{\mathcal{D}^k\}_{1 \leq k \leq N}$

end function

Policy initialization

Setting mixing parameters (mixing average strategy and best response profiles) and generating datasets

Updating RL/SL memory and training average strategy of j iterations and best response strategy for $j+1$ iteration by using SL and RL respectively

Mixing strategies and generating n episodes from mixed strategy and m episodes from best response strategy for each player

Multi-agent learning

- **FSP**: Avoiding cycles by computing a best response against a uniform mixture of all previous policies.
- However, **FSP** wastes time on players that are defeated.
- Therefore, **PFSP** is introduced, which extends FSP to compute a best response against a non-uniform mixture of opponents.

Multi-agent learning

- **PFSP:** Prioritized fictitious self-play
- In **PFSP**, agent A is expected to sample opponents by using following possibility, which makes PFSP focus on the hardest players or players around its level:

$$\frac{f(P[A \text{ Beats } B])}{\sum_{c \in C} f(P[A \text{ Beats } C])}, f_{hard} = (1 - x), f_{var} = x(1 - x)$$

Multi-agent learning

- SP – FSP – PFSP:

Self-play Types	Opponents	Distribution	Solving
Self-play	Current policy	-	Naive self-play: Chase cycles.
Fictitious self-play	All previous policies	Uniform	Avoiding cycles but wasting time on defeated opponents.
Prioritized fictitious self-play	All previous policies	Non-uniform	Focusing on choose the hardest players or players around its level.

Multi-agent learning

- **Populating the league:** (for one race)

Agent Types	Num	Snapshot	Reset	Function
Main agents	1	Every 2×10^9 steps.	Never.	AlphaStar agents.
League exploiters	2	Defeat all players or 2×10^9 steps.	25% reset after adding to leagues.	Identify global blind spots in the league.
Main exploiters	1	All three main agents are defeated or 4×10^9 steps.	100% reset after adding to leagues.	Identify weakness of main agents.

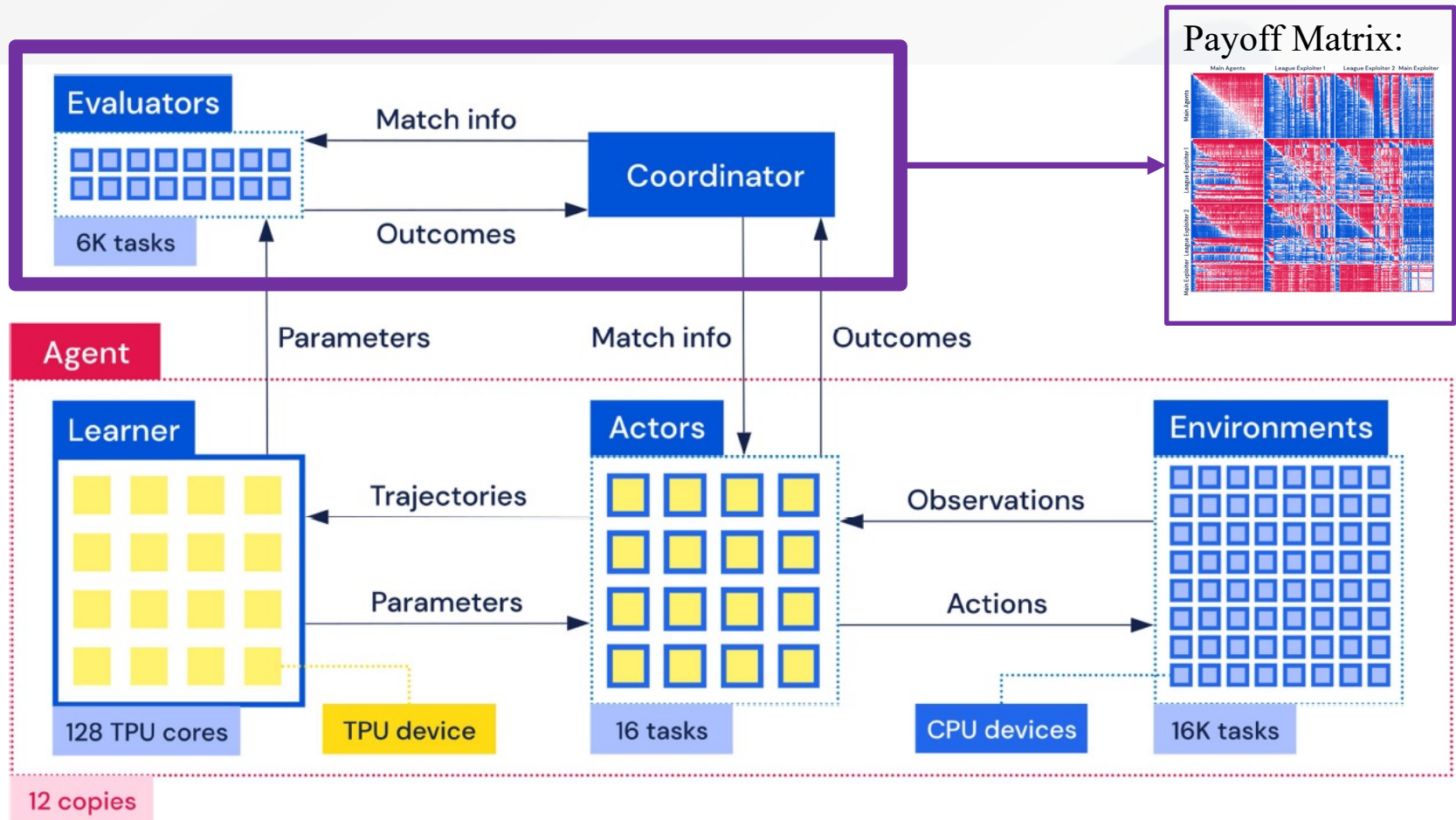
Multi-agent learning

- **Populating the league:** (for one race)

Agent Types	Distribution
Main agents	35% SP + 50% PFSP against all past players in the league + 15% PFSP against forgotten undefeated main players and past main exploiters. Or: 50% SP + 50% PFSP if no forgotten players or strong exploiters.
League exploiters	100% PFSP against all players in the league.
Main exploiters	If win rate is low: PFSP with f_{var} weighting against players created by the main agents. Otherwise: PFSP against current main agents.

Evaluators

- Coordinator and evaluators: re-evaluate the internal payoff estimation



Evaluation and analysis

- To evaluate the win rates between players, introduce **Internal Elo ratings**, We added the built-in bots, and used them to estimate Elo with the following model:

$$P[r_1 \text{ Beats } r_2] = \frac{1}{1 + e^{-(r_1 - r_2)/400}} \approx \phi\left(\frac{r_1 - r_2}{400}\right)$$

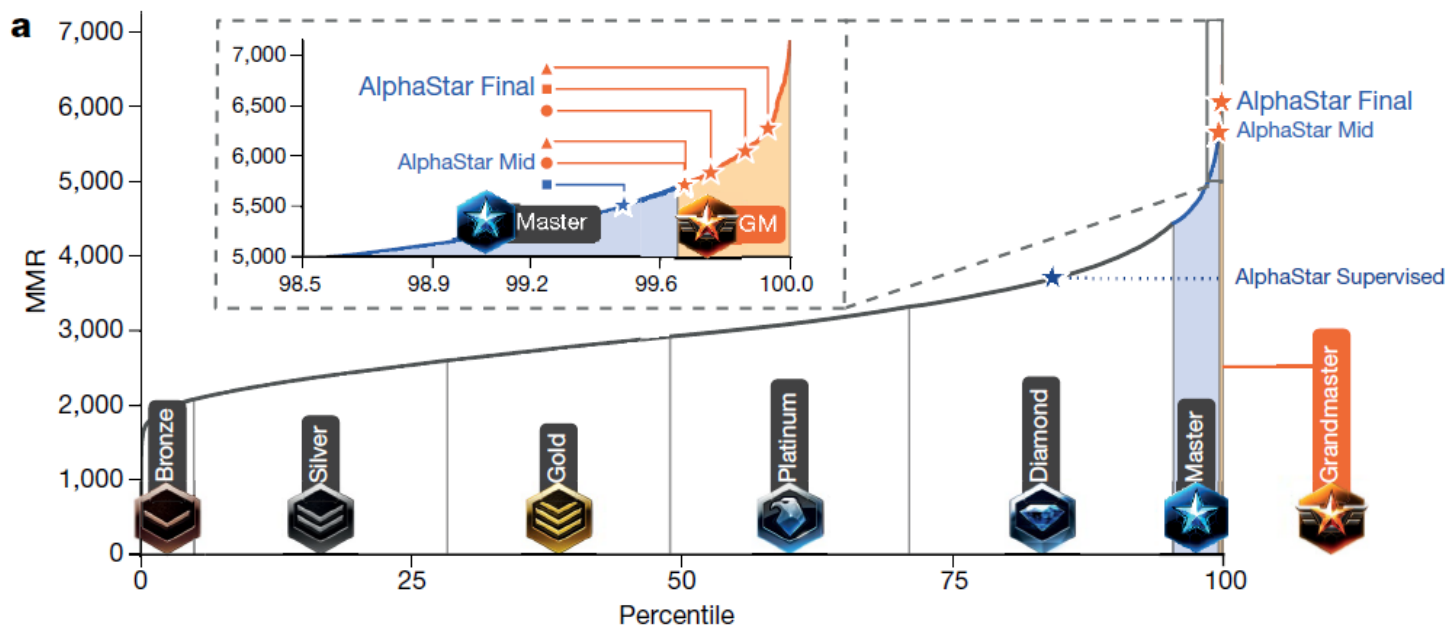
- RPP:** Expected outcome of the meta-game between 2 populations:
- Given payoff matrix between all agents in leagues A and B of size N and M:

$$RPP(P_{AB}) = \text{Nash}(P_{AB})^T P_{AB} \text{Nash}(P_{BA}), P_{AB} \in [0,1]^K$$

Outline

- Introduction
- Related works
- Algorithm details
- **Experiments and Results**
- Conclusion

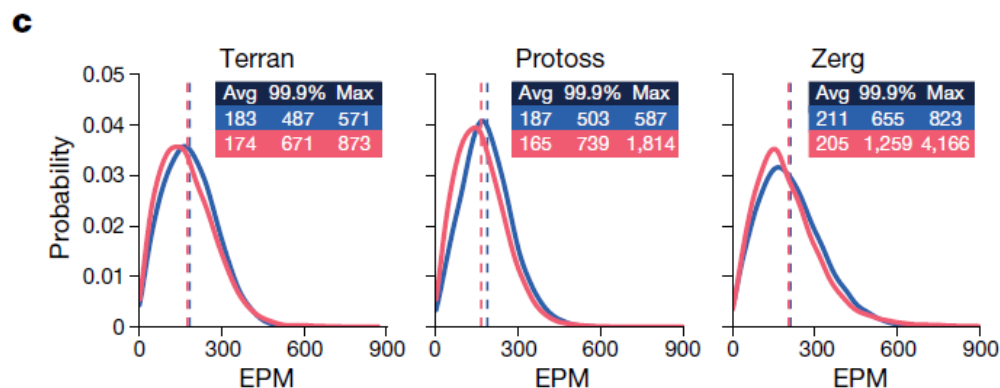
Overall rating



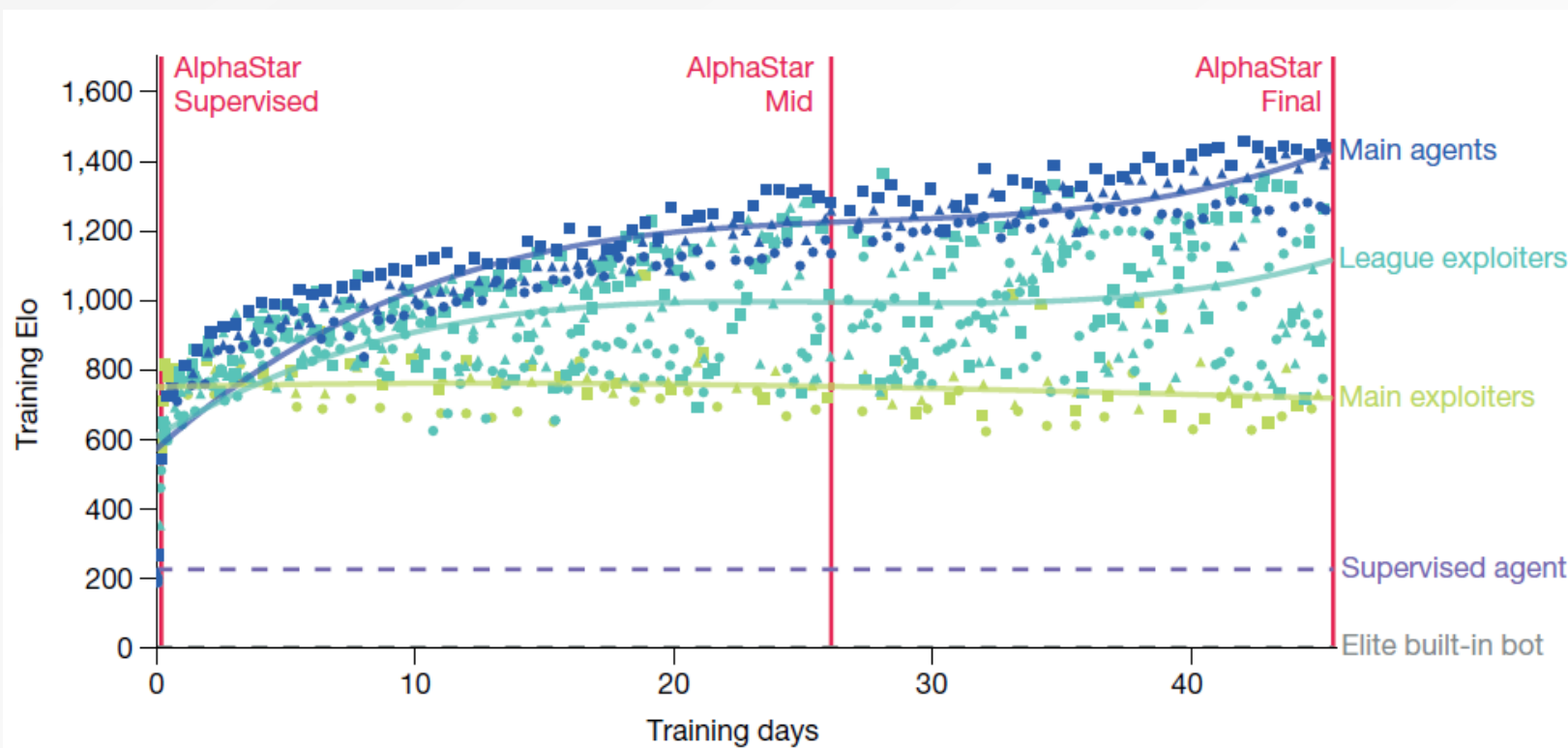
b

Opponent race

AlphaStar race	Protoss	Zerg	Human	AI
Protoss	6,275 99.93% 25/30	6,196 99.91% 11/14	— — 4/4	6,297 99.94% 10/12
Zerg	6,048 99.86% 18/30	5,991 99.83% 4/8	6,209 99.92% 4/7	5,971 99.82% 10/15
Human	5,835 99.78% 18/30	5,755 99.7% 8/14	5,531 99.51% 5/10	6,500 99.96% 5/6

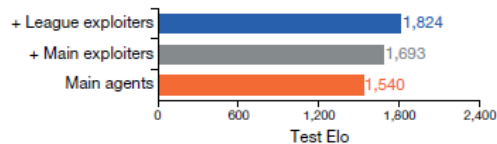


Training progression

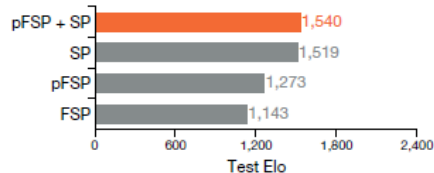


Ablation studies

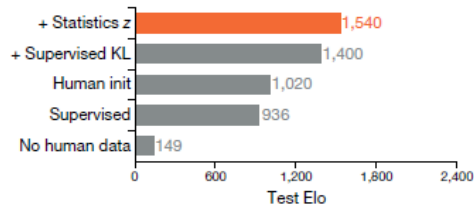
a League composition



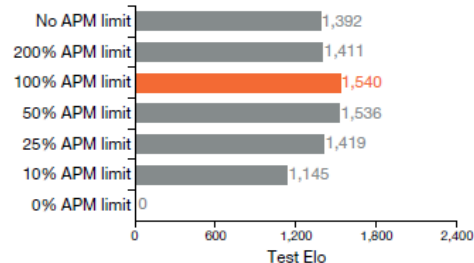
c Multi-agent learning



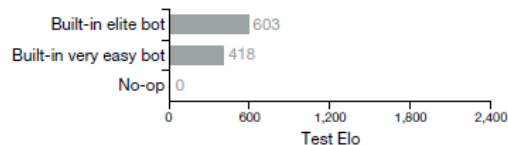
e Human data usage



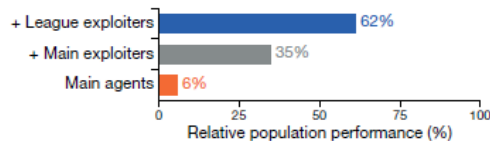
g APM limits



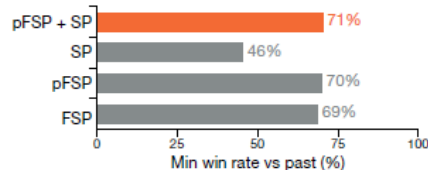
j Bots baselines



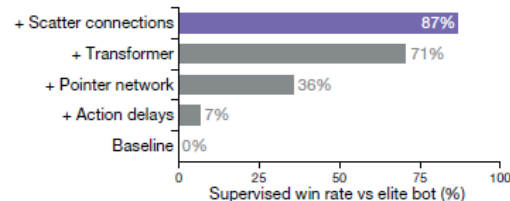
b League composition



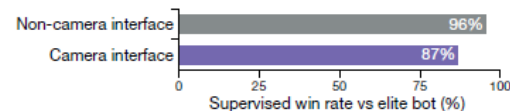
d Multi-agent learning



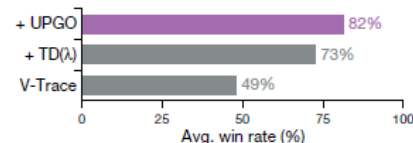
f Architectures



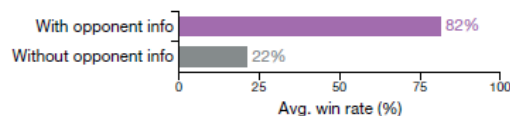
h Interface



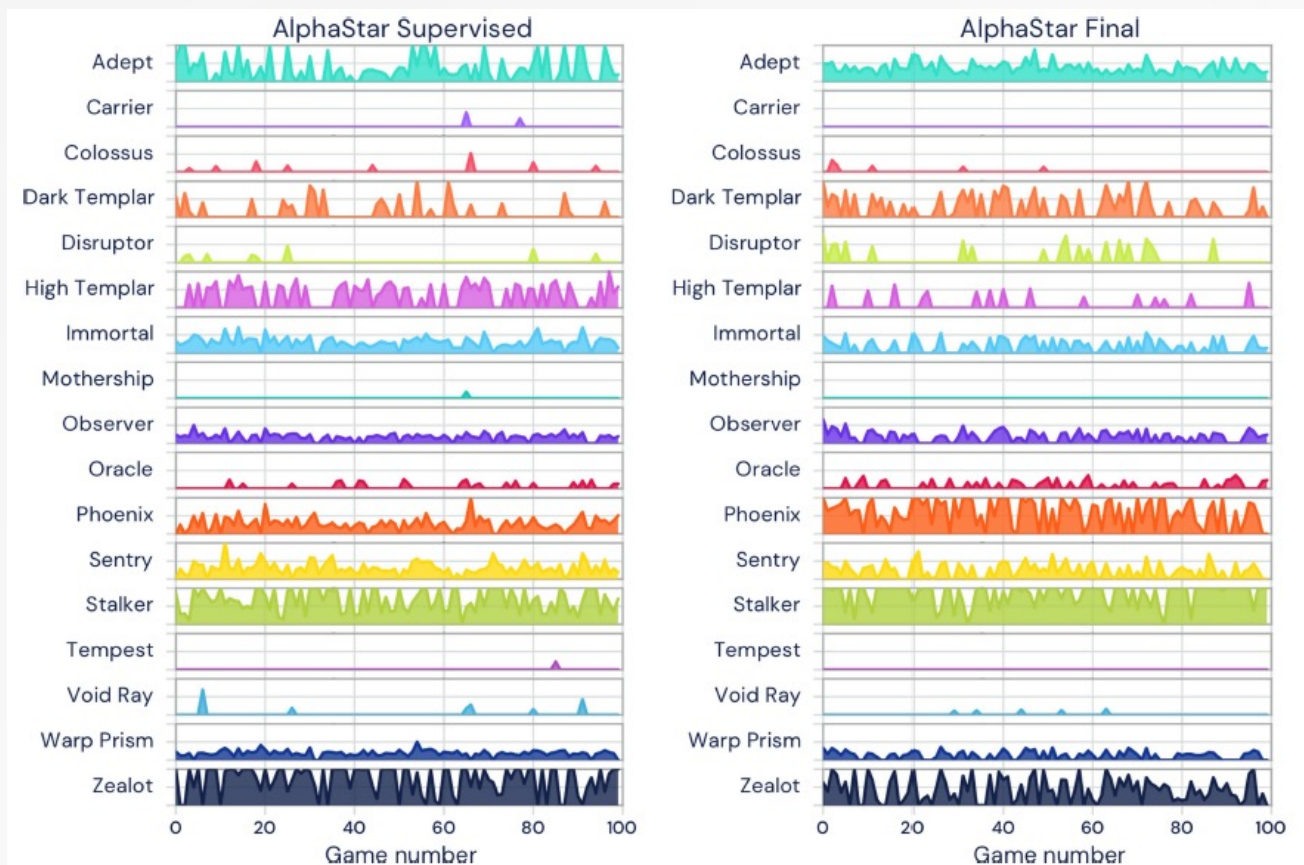
i Off-policy learning



k Value function



Distribution of units



Distribution of units built in a game. Units built by Protoss AlphaStar Supervised (left) and AlphaStar Final (right) over multiple self-play games.

Outline

- Introduction
- Related works
- Algorithm details
- Experiments and Results
- Conclusion

Conclusion

- AlphaStar is the first agent to achieve **Grandmaster level** in StarCraft II, and the first to reach the highest league of human players in a widespread professional e-sport without simplification of the game.
- The success of AlphaStar in StarCraft II suggests that **general-purpose machine learning algorithms** may have a substantial effect on complex real-world problems.



Thanks !

CASIA